

STREAMING REINFORCEMENT LEARNING UNDER PARTIAL OBSERVABILITY WITH REAL-TIME RECURRENT LEARNING

Anonymous authors

Paper under double-blind review

ABSTRACT

We show that exact real-time recurrent learning (RTRL) over a diagonal recurrence composes with the eligibility trace of a streaming reinforcement learning update, giving streaming agents a credit horizon bounded by the slower of the trace decay and the retention of the recurrence. Streaming methods learn from one transition at a time with no replay buffer, which has kept partially observable tasks out of reach, since truncated backpropagation collapses to a one-step horizon in that setting and RTRL for general recurrent networks is quartic in the hidden size. Restricting to recurrences with a diagonal state-to-state Jacobian, satisfied by RTUs, minGRUs or any other diagonal state-space model, makes exact RTRL linear in the parameter count and lets it slot into any streaming algorithm built on a per-step gradient and an eligibility trace. The composition predicts which quantity sets the decay of credit under each gradient computation, and measurements on trained agents separate them: lowering the trace decay moves the truncated weight and leaves the exact one where the retention puts it. On trace conditioning and MemoryChain exact RTRL tracks horizons up to four times longer than one-step truncation, and under the intentional update it solves chain length 256 at every trace decay we test, including the standard 0.8 where the eligibility trace spans five steps, which one-step truncation reaches only as λ approaches one. On eleven POPGym tasks the same agents beat their own memoryless counterparts on six to nine depending on the algorithm, and on masked MuJoCo they reach parity with recurrent PPO at matched capacity, indistinguishable on five of eight task-regime cells, once the trace decay is set for the recurrence rather than inherited from feedforward work.

1 INTRODUCTION

We show that exact real-time recurrent learning (Real-Time Recurrent Learning (RTRL)) over a diagonal recurrence composes with the eligibility trace of a streaming reinforcement learning update, and that the composition sets the horizon over which a streaming agent can assign credit. Streaming agents update from each transition as it arrives, with no replay buffer and constant memory (Elsayed et al., 2024; Vasan et al., 2024), but so far they have been confined to fully observable tasks, because backpropagation through time collapses to a one-step horizon, TBPTT(1), when there is no stored trajectory to unroll.

RTRL (Williams & Zipser, 1989) maintains the recurrent gradient forward in time and is the natural fit for streaming, but its cost is quartic in the hidden size for general recurrences. It becomes linear in the parameter count when the state-to-state Jacobian is diagonal, since each parameter then influences only its own unit. Recurrent Trace Units (RTU)s (Elelimy et al., 2024), minGRUs (Feng et al., 2024) and diagonal state-space models (Orvieto et al., 2023) all satisfy this, where a Gated Recurrent Unit (GRU) (Cho et al., 2014) or Long Short-Term Memory (LSTM) (Hochreiter & Schmidhuber, 1997) does not. Prior work has trained such cells with Proximal Policy Optimization (PPO) (Schulman et al., 2017; Elelimy et al., 2024) or with approximate forward-mode gradients inside a TD(λ) actor-critic (Lemmel & Grosu, 2025). We insert exact RTRL into existing streaming algorithms unchanged and analyse the resulting update. The RTRL sensitivity trace and the eligibility trace form one triangular linear system whose slower mode, the larger of the trace decay $\gamma\lambda$ and the retention

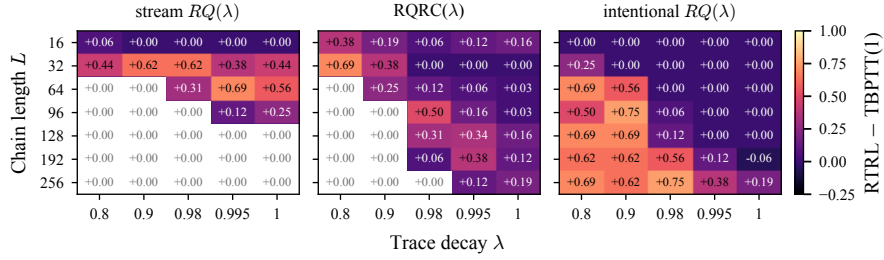


Figure 1: Where the composition predicts exact RTRL helps, and where it does not. Gain in solve rate from exact RTRL over TBPTT(1) on MemoryChain, as a function of chain length L and trace decay λ , for three streaming algorithms with a minGRU recurrence ($\gamma=0.999$, 16–32 seeds per cell). A run counts as successful when its final return exceeds 0.5. White cells mark combinations where neither method solves the task, distinguishing them from cells where both already succeed and the difference is likewise zero. Per-method solve rates for all three sweeps are given in Appendix A.

ρ of the recurrence, sets the rate at which credit on past steps decays, while TBPTT(1) decays at $\gamma\lambda$ alone.

The evidence is of three kinds. Measurements on trained agents show the truncated weight tracking $\gamma\lambda$ while the exact weight does not, and that retention is learned by minGRU but supplied by RTU’s initialisation. On trace conditioning (Figure 2) and MemoryChain, exact RTRL tracks horizons up to four times longer than TBPTT(1), and on MemoryChain the reachable chain length grows with λ under one-step truncation while exact RTRL extends it further at every λ , by one grid step under RQRC(λ) and to $L=256$ independently of λ under the intentional update. On eleven POPGym tasks the recurrent agents beat the identical updates without a recurrence on six to nine depending on the algorithm, on masked MuJoCo they learn continuous control from single transitions and reach parity with recurrent PPO, and on Foragax they keep relearning under a flipping reward. We do not claim state-of-the-art performance on any single benchmark.

Contributions.

- A proof that the RTRL sensitivity trace and the eligibility trace form one triangular system whose slower mode bounds the credit horizon, with measured credit weights separating the two channels on trained agents.
- The consequence that horizon is not shared: parameters reached by the sensitivity inherit the recurrence’s retention, while the rest of the network decays at $\gamma\lambda$, so λ remains consequential even when the recurrence retains for hundreds of steps. Neither λ nor the streaming step-size rules transfer from feedforward work, and we show why: the rules assume the trace is a decayed sum of recent gradients, which RTRL violates in three distinct ways (Appendices F and E).
- Streaming deep RL for partial observability with exact RTRL over any diagonal-Jacobian recurrence, instantiated with RTUs and minGRUs in five streaming algorithms, evaluated on trace conditioning, MemoryChain, POPGym, masked MuJoCo and continual foraging against recurrent PPO at matched capacity, with a staleness bound for the sensitivity matrix under streaming updates in Appendix N.

2 BACKGROUND

Streaming reinforcement learning with eligibility traces. A streaming agent updates from each transition and discards it, and propagates credit across time with eligibility traces (Sutton & Barto, 1998): for a per-step gradient g_t of a value function or policy with respect to parameters w_t ,

$$z_t = \gamma\lambda z_{t-1} + g_t, \quad w_{t+1} = w_t + \alpha\delta_t z_t, \quad z_0 \doteq \mathbf{0}, \quad (1)$$

where δ_t is the TD error, α the step size and $\lambda \in [0, 1]$ the trace decay. Existing streaming deep RL methods (Elsayed et al., 2024; Vasan et al., 2024) use feedforward function approximation, where g_t is the backpropagated gradient of a single sample.

Real-time recurrent learning. For a recurrent network $\mathbf{h}_t = \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\psi})$ with parameters $\boldsymbol{\psi}$, computing $\mathbf{g}_t$ requires the gradient $\partial \mathbf{h}_t / \partial \boldsymbol{\psi}$ of the current hidden state with respect to the recurrent parameters. Backpropagation through time (BPTT) obtains this gradient by unrolling the recurrence backward over a stored trajectory of length T , which reduces to a one-step gradient horizon when $T = 1$. RTRL (Williams & Zipser, 1989) avoids this unrolling by maintaining the gradient forward through time, applying the chain rule to the recurrence,

$$\frac{\partial \mathbf{h}_t}{\partial \boldsymbol{\psi}} = \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\psi})}{\partial \boldsymbol{\psi}} + \frac{\partial \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\psi})}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \boldsymbol{\psi}}. \quad (2)$$

The first term is the direct dependence of $\mathbf{h}_t$ on $\boldsymbol{\psi}$ through the current step. The second propagates accumulated dependence through $\mathbf{h}_{t-1}$. For a general recurrent network with hidden dimension n and parameter count $O(n^2)$, this gradient is an $n \times O(n^2)$ matrix and the recursion costs $O(n^4)$ per step, prohibitive even for moderate n .

Diagonal recurrences. RTUs (Elelimy et al., 2024) sidestep this barrier with a linear diagonal complex-valued recurrence (their linear variant, with a nonlinearity only on the output),

$$\mathbf{h}_t = \boldsymbol{\Lambda} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{x}_t, \quad (3)$$

where $\boldsymbol{\Lambda} \in \mathbb{C}^{n \times n}$ is diagonal with entries $\lambda_k = r_k(\cos \Omega_k + i \sin \Omega_k)$, where r_k and Ω_k define the polar complex representation of the k 'th parameter. Because each hidden unit has only one recurrent connection to itself, exact RTRL has linear time and memory complexity in the number of parameters, which Figure 9 in Appendix C confirms empirically.

The minGRU (Feng et al., 2024) satisfies the same condition through a real gate rather than a complex rotation: $\mathbf{h}_t = (\mathbf{1} - \mathbf{u}_t) \odot \mathbf{h}_{t-1} + \mathbf{u}_t \odot \mathbf{W}_h \mathbf{x}_t$ with $\mathbf{u}_t = \sigma(\mathbf{W}_u \mathbf{x}_t)$ depending on the input alone, so $\mathbf{J}_t = \text{diag}(\mathbf{1} - \mathbf{u}_t)$ and exact RTRL is again linear in the parameter count. The two differ in where retention comes from, RTU's r_k being set near one at initialisation while the minGRU gate starts at 0.5 and must raise it, and in how credit accumulates, since a real positive retention adds terms that cannot cancel while RTU's complex rotation can interfere destructively across lags (Section 3).

3 METHODOLOGY

Our method makes a single change to existing streaming algorithms. Where an algorithm typically uses a feedforward function approximator mapping observations to values or actions, we insert an RTU layer between the observation and the feedforward head. The streaming update machinery, including eligibility traces and step-size adaptation, is unchanged. The only modification is how the per-step gradient that feeds into the eligibility trace is computed. Figure 14 shows the data flow, and we describe the gradient computation for a single network below.

Per-step gradient. The function approximator has three parameter groups: an encoder $\phi(\cdot; \boldsymbol{\theta})$ producing features $\mathbf{x}_t$, a recurrent RTU layer with parameters $\boldsymbol{\psi}$ producing the state $\mathbf{h}_t$, and a feedforward head with parameters $\mathbf{w}$ (a value head $v(\mathbf{h}_t; \mathbf{w}_v)$, say). We compute the gradient of the head output with respect to each group in a single backward pass, combined with the RTU's forward-mode RTRL trace $\mathbf{S}_t = \partial \mathbf{h}_t / \partial \boldsymbol{\psi}$.

The head parameters have no temporal dependency and receive an ordinary backward pass. The recurrent parameters enter only through $\mathbf{h}_t$, so combining the backward pass through the head with $\mathbf{S}_t$ yields their gradient across the entire history at $\mathcal{O}(|\boldsymbol{\psi}|)$ per step. The encoder parameters are more complicated. They influence the state both instantaneously through $\mathbf{x}_t$ and recursively through every past state. Maintaining the exact trace would cost $\mathcal{O}(|\boldsymbol{\theta}| \cdot \dim \mathbf{h})$ per step, prohibitive for a deep encoder. We therefore use a one-step approximation, backpropagating through the head, a single step of the recurrence, and the encoder. This credits the encoder for its immediate effect on $\mathbf{h}_t$ while ignoring its influence through past states.

Composition with eligibility traces. Operationally the two traces compose without modification. The RTRL trace is a forward-mode object that maintains $\partial \mathbf{h}_t / \partial \boldsymbol{\psi}$ at each step; the eligibility trace is a temporal accumulator of whatever per-step gradient the streaming algorithm supplies. Each

RTU layer supplies its $\partial \mathbf{h}_t / \partial \psi$ on demand whenever the streaming update needs the gradient with respect to its recurrent parameters, and the eligibility trace folds this into its usual recursion.

Their effects on credit assignment, however, are not independent. Both carry credit backward in time, and the reach of the update is bounded by whichever decays more slowly. The bound governs how credit is weighted, not whether any is available to assign. Appendix L.4 gives three qualifications, including why λ is not free and why $\rho > \gamma\lambda$ is the typical case.

Proposition 1. *Let $\Phi_{i,k} \doteq \mathbf{J}_i \mathbf{J}_{i-1} \cdots \mathbf{J}_{k+1}$ for $i > k$, with $\Phi_{k,k}$ the identity matrix. At the end of an episode of length T , both updates are weighted sums over the same immediate Jacobians, $\mathbf{z}_T = \sum_{k=0}^T \omega_k \mathbf{I}_k$, with*

$$\omega_k^{\text{TBPTT}(1)} = (\gamma\lambda)^{T-k} \mathbf{c}_k^\top, \quad \omega_k^{\text{RTRL}} = \sum_{i=k}^T (\gamma\lambda)^{T-i} \mathbf{c}_i^\top \Phi_{i,k}. \quad (4)$$

Writing $\rho \doteq \max_t \|\mathbf{J}_t\|$ and $C \doteq \max_t \|\mathbf{c}_t\|$,

$$\|\omega_k^{\text{RTRL}}\| \leq C \sum_{i=k}^T (\gamma\lambda)^{T-i} \rho^{i-k} = \mathcal{O}(\max(\gamma\lambda, \rho)^{T-k}) \quad (5)$$

for $\rho \neq \gamma\lambda$; at equality the sum has $T - k + 1$ terms of equal size and the bound carries an additional factor $T - k + 1$.

Substituting the RTRL recursion into the eligibility trace shows why: $(\mathbf{S}_t, \mathbf{z}_t)$ evolves as one block-triangular linear system whose modes are those of the recurrence together with $\gamma\lambda$, and TBPTT(1) severs the coupling between them, so the decay is bounded by whichever mode is slower. Writing $n = T - k$ for the lag, the ratio of the two weights on the same immediate Jacobian separates into two regimes,

$$\frac{\|\omega_k^{\text{RTRL}}\|}{\|\omega_k^{\text{TBPTT}(1)}\|} \sim \begin{cases} n + 1, & \rho \approx \gamma\lambda, \\ \left(\frac{\rho}{\gamma\lambda}\right)^n, & \rho > \gamma\lambda, \end{cases} \quad (6)$$

since the geometric sum collapses to $n + 1$ equal terms when the modes coincide and is dominated by its last otherwise. Exact RTRL therefore amplifies distant credit linearly or exponentially in the lag, and offers no advantage in decay rate only when $\rho < \gamma\lambda$. We measure the rates directly below.

The two channels separate in measurement. Freezing a trained agent and measuring $\|\omega_k\|$ against lag on MemoryChain ($L=64$, medians over 10 seeds), the TBPTT(1) weight decays at $\gamma\lambda$ to within 0.2% in every configuration, while the RTRL weight does not follow $\gamma\lambda$: dropping λ from 1 to 0.98 moves the fitted TBPTT(1) rate from 0.999 to 0.979 and leaves the RTRL rate at 0.994 or above. Over 32 lags that rate sits at or just above one, the $n + 1$ prefactor of the near-degenerate regime $\rho \approx \gamma\lambda$ rather than a decay, so the bound orders the two channels without being tight on either. The amplification at lag 32 is cell-dependent, 20 to 33 for minGRU against 6 to 9 for RTU, which the same measurements attribute to phase cancellation: minGRU’s coherence is exactly 1 at every lag because its retentions are real and positive, while RTU’s complex rotation drives it to 0.23–0.46 by lag 32. What the measurements do not establish is sufficiency, since the credit horizon exceeds the observed solvable chain in every cell of every grid. Appendix B.1 gives the protocol, the fitted rates and the effect of exploration cuts, which favour RTRL because zeroing the eligibility trace on a non-greedy action leaves the influence matrix intact.

Instantiations. We evaluate on five streaming algorithms covering prediction, value-based control and policy-based control: stream TD(λ), stream Q(λ) and stream AC(λ) (Elsayed et al., 2024), QRC(λ) (Elelimy et al., 2025) and intentional Q(λ) (Sharifnassab et al., 2026). We call their recurrent instances stream RTD(λ), stream RQ(λ), stream RAC(λ), RQRC(λ) and intentional RQ(λ), and the TBPTT(1) ablations use the same networks with the one-step gradient. Stream RTD(λ) is applied to trace conditioning, the three value-based algorithms to MemoryChain and POPGym, and stream RAC(λ) and intentional RAC(λ) to masked MuJoCo. Each network carries its own recurrent layer and RTRL trace, so their recurrent parameters are updated independently. Appendix K gives pseudocode.

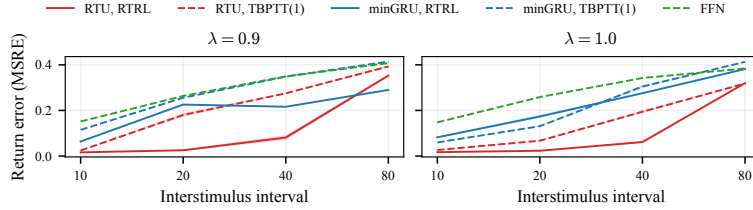


Figure 2: Return error on trace conditioning against the interstimulus interval, IQM with standard error over 10 seeds, for stream $\text{RTD}(\lambda)$ under exact RTRL (solid) and TBPTT(1) (dashed) at $\lambda=0.9$ and $\lambda=1$, with γ per ISI as in Rafiee et al. (2023). At $\lambda=0.9$ exact RTRL with RTU tracks the timing where TBPTT(1) falls to the memoryless baseline, and at $\lambda=1$ the trace carries TBPTT(1) most of the way (Section 4.1).

4 EXPERIMENTS

We evaluate the method on five benchmarks chosen to isolate distinct properties, and on a sixth in Appendix M. Trace conditioning (Section 4.1) tests the credit horizon on pure prediction, and MemoryChain (Section 4.2) sweeps it under control. POPGym (Section 4.3) tests whether the streaming method is competitive with a recurrent baseline that reuses each transition, on long-memory discrete control. Masked MuJoCo (Section 4.4) extends the evaluation to continuous control. Foragax (Section 4.5) tests continual relearning under a periodically inverted reward. All curves report the interquartile mean (IQM) with shaded standard error, and seed counts vary by experiment and are stated in each caption. The recurrence is not held constant: each benchmark uses the cell from the work it comes from, minGRU for MemoryChain and RTU elsewhere, and Appendix H repeats POPGym with the other two. Hyperparameters, network architectures, and per-task details are listed in Appendix J.

4.1 TRACE CONDITIONING

Exact RTRL tracks interstimulus intervals that one-step truncation cannot, and λ matters far more for the truncated update than for the exact one.

Task and design. Trace conditioning (Rafiee et al., 2023) is a prediction task: a conditioned stimulus precedes an unconditioned one by an interstimulus interval (ISI), and the agent observes both and predicts the discounted sum of the unconditioned stimulus, with γ set per ISI as in the original benchmark so that $1/(1-\gamma)$ matches the ISI. This is the setting of Proposition 1: prediction under a fixed behaviour, with the memory length set by the ISI. We run stream $\text{RTD}(\lambda)$ with RTU, minGRU and a memoryless network for 2M steps and 10 seeds and report the mean squared error of the prediction against the true return.

Result. Figure 2 shows that RTU under exact RTRL is the only configuration that tracks the timing beyond ISI 10: at ISI 20 and 40 its error is $5.1\times$ and $3.1\times$ below TBPTT(1) at $\lambda=0.9$, where TBPTT(1) sits near the memoryless baseline. Raising λ to 1 helps TBPTT(1) substantially (0.18 to 0.07 at ISI 20), which is the eligibility trace extending the horizon. Exact RTRL moves less but not negligibly: at ISI 40 its error falls from 0.090 at $\lambda=0.9$ to 0.059 at 0.95, rising again to 0.064 at 0.99 and 1, so an interior optimum exists here as it does on masked MuJoCo, at a different λ (Appendix F). At ISI 80 the recurrent configurations are at best level with the memoryless baseline within 2M steps, several of them sitting just below it in error, and minGRU trails RTU at every ISI except 80.

4.2 MEMORYCHAIN

The solvable chain length grows with the trace decay under TBPTT(1). Exact RTRL reaches further at every decay, and under the intentional update it stops depending on the decay at all.

Task and design. The MemoryChain (Osband et al., 2020) task presents an informative binary cue at $t=0$. The agent then receives uninformative observations for $L-1$ steps and is rewarded for reproducing the cue at $t=L$. The chain length L controls how far back the agent must propagate credit, isolating the temporal credit-assignment problem from representation learning. We sweep the chain length and the trace decay jointly, $L \in \{16, 32, 64, 96, 128, 192, 256\}$ against $\lambda \in \{0.8, 0.9, 0.98, 0.995, 1\}$, and run all three streaming algorithms, stream $RQ(\lambda)$, $RQRC(\lambda)$ (El elimy et al., 2025) and intentional $RQ(\lambda)$, each with a minGRU recurrence under exact RTRL and under TBPTT(1), at $\gamma=0.999$ with 16–32 seeds per cell. A run counts as successful when its final return exceeds 0.5, and all non-recurrence hyperparameters are matched across agents.

The trace decay controls this gap. Proposition 1 predicts that this comparison is specific to the regime $\rho > \gamma\lambda$, since the eligibility trace and the RTRL trace carry credit in parallel and the slower mode dominates. At $\lambda=0.8$ and $\gamma=0.99$ the trace mode decays with an effective horizon of roughly five steps, which is consistent with the collapse observed past $L=16$; the recurrent gradient is then the only channel that remains. Raising λ to one removes that limit. Figure 1 sweeps both quantities jointly under otherwise identical settings ($\gamma=0.999$, minGRU recurrence, 16–32 seeds per cell) for three streaming algorithms. Section 3 measures $\rho \approx 0.999$ on this task, a retention horizon of roughly a thousand steps and longer than any chain in the sweep, so the prediction is that the TBPTT(1) boundary moves with λ while the RTRL boundary need not. Under $RQRC(\lambda)$ the longest chain TBPTT(1) solves on a majority of seeds grows with the trace horizon $(1 - \gamma\lambda)^{-1}$, from $L=16$ at $\lambda=0.8$ to $L=128$ at $\lambda=0.995$, though sublinearly: the ratio $L/(1 - \gamma\lambda)^{-1}$ falls from 3.2 to 0.77 across the sweep. Exact RTRL moves that boundary one grid step deeper at every λ but one, and its advantage is largest along the boundary, reaching 0.69 at $L=32$, $\lambda=0.8$. The advantage vanishes in two limits the figure keeps distinct: at high λ the trace already spans the episode and the two channels coincide, while at long L neither method succeeds. We do not attribute the latter to the recurrence losing the cue, since the same minGRU solves $L=256$ under the intentional update at every λ .

The ordering is algorithm-independent and the reachable horizon is not. Exact RTRL is at least as good as TBPTT(1) in every cell of all three grids but one, with mean gains of +0.13, +0.14 and +0.25, while the absolute reach differs by an order of magnitude: stream $RQ(\lambda)$ never passes $L=64$, whereas under intentional $RQ(\lambda)$ exact RTRL solves $L=256$ at every λ including 0.8, where the trace spans five steps. That λ -independence is what Proposition 1 predicts for the RTRL channel once ρ exceeds $\gamma\lambda$. The spread tracks the retention each algorithm learns, 0.981, 0.999 and 0.99995 respectively, ordered exactly as their reachable horizons are. It is not sufficient, however: the credit horizon $(1 - \rho)^{-1}$ exceeds the observed boundary in every cell of every grid, by factors from 1.5 to nearly 400. The choice of cell matters independently of the gradient method, and freezing the recurrence at initialisation solves $L=16$ and nothing beyond $L=32$. Appendix A gives the full grids, the per-cell gains and the frozen control.

4.3 POPGYM

The recurrence earns its place on six to nine of eleven tasks depending on the algorithm, and the streaming methods trail recurrent PPO on eight of eleven, where reusing transitions pays. Masked MuJoCo reaches parity once the trace decay is set for the recurrence.

Task and design. We evaluate on eleven tasks from the POPGym Easy suite (Morad et al., 2023): Autoencode, Battleship, Concentration, CountRecall, HigherLower, MineSweeper, MultiArmed-Bandit, NoisyStatelessCartPole, RepeatFirst, RepeatPrevious, and StatelessCartPole. We compare three streaming value-based methods, stream $RQ(\lambda)$, $RQRC(\lambda)$ and intentional $RQ(\lambda)$, each paired with RTU, against a recurrent PPO-RTU baseline in the real-time design of Elelimy et al. (2024), which stores the RTRL sensitivity of every rollout step and updates on single-step minibatches, so every sample carries an exact recurrent gradient. It uses the same recurrence without the streaming constraint. Each agent is run for 10M environment frames with 10 seeds, and we report the interquartile mean with shaded standard error. Returns are normalised per task so that 0 is the minimum episodic return and 1 the return of an optimal policy, which puts the panels on a common scale.

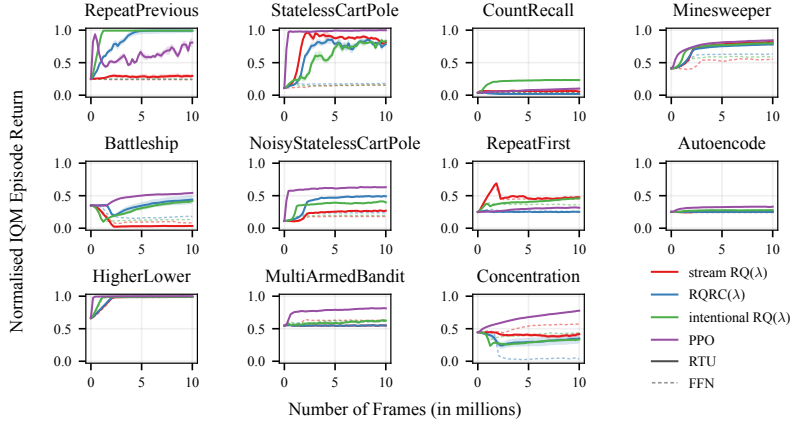


Figure 3: IQM episodic return over 10 seeds, normalised per task so that 0 is the minimum return and 1 the return of an optimal policy, with standard error on eleven POPGym tasks, and each curve starts at the return of the freshly initialised policy. Solid curves use an RTU recurrence, dashed curves the same algorithm with a feedforward network.

Memoryless ceiling. POPGym aliases the previous action into the observation, so a memoryless agent sees (o_t, a_{t-1}) . Each algorithm is plotted twice, solid with an RTU recurrence and dashed with a feedforward network (FFN), at the same budget and seed count, so every comparison is against the identical update with the recurrence removed. Bootstrapping that difference at 10M frames, the recurrence is ahead on six of eleven tasks under stream $RQ(\lambda)$, six under $RQRC(\lambda)$ and nine under intentional $RQ(\lambda)$, and behind on two, four and one respectively, the rest unresolved.

Result. The ceiling partitions the suite. On three of the eleven tasks no streaming method with RTU clears it: HigherLower is reached exactly at the ceiling, and on MultiArmedBandit and Concentration the best streaming method reaches 0.63 and 0.42 against ceilings of 0.67 and 0.66. Recurrent PPO clears all three. On these the agents fail to recover behaviour a reactive baseline already implements, so they say little about recurrent architectures. On the remaining eight at least one streaming method exceeds the ceiling, though on Autoencode every method stays within 0.03 of initialisation.

Against recurrent PPO the streaming methods do not reach parity. They finish ahead on three tasks, all of them recall, where a single pass over a long-horizon dependency suffices: RepeatPrevious (0.99 against 0.81), RepeatFirst (0.48 against 0.31) and CountRecall (0.23 against 0.10). They finish behind on the other eight, narrowly on Minesweeper, Autoencode and HigherLower, where every method sits at the ceiling, and by a wide margin on Concentration (0.42 against 0.78), MultiArmedBandit (0.63 against 0.81) and both cartpole variants, where reusing each transition many times pays. Intentional $RQ(\lambda)$ is the strongest streaming method here, ahead of stream $RQ(\lambda)$ on seven of eleven tasks and behind on two by at most 0.08, with its largest margins on Battleship and RepeatPrevious, where stream $RQ(\lambda)$ reaches 0.03 and 0.30 against its 0.42 and 0.99. $RQRC(\lambda)$ is the weakest.

4.4 MASKED MUJoCo

Task and design. Following Ni et al. (2022) we evaluate on Ant, HalfCheetah, Hopper and Walker2d under masked velocities (**P**) and masked positions (**V**). We compare stream $RAC(\lambda)$ and intentional $RAC(\lambda)$ (Sharifnassab et al., 2026), both with RTU, against recurrent PPO with RTU, using 5M environment frames and 10 seeds per run. So that the comparison is not confounded by capacity, PPO is run with the same network as the streaming agents rather than the one reported by Elelimy et al. (2024), giving 139,405 parameters against 141,709. We set $\lambda = 0.99$ rather than the 0.8 used by Elsayed et al. (2024) and Sharifnassab et al. (2026), for the reason given in Appendix F: their value was selected on fully observed control, where no part of the network has to integrate an observation stream.

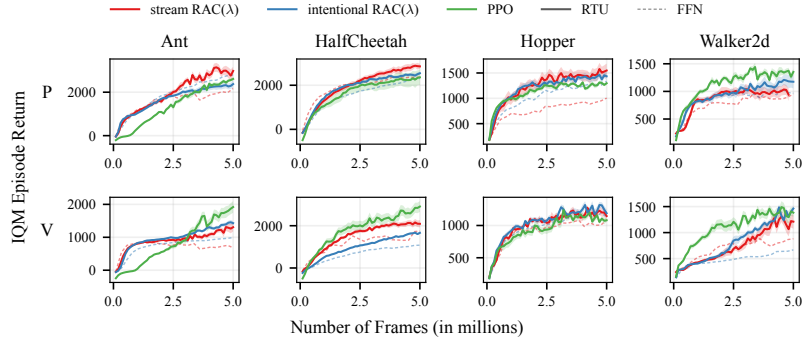


Figure 4: IQM episodic return with standard error, 10 seeds, on four MuJoCo tasks under masked velocities (P) and masked positions (V). Streaming methods use $\lambda = 0.99$, and PPO uses the same network as the streaming agents (Appendix J). Solid curves use an RTU recurrence, dashed curves the same algorithm with a feedforward network. Intentional RAC(λ) uses the trace-estimated preconditioner and $\eta_\pi = 0.01$ (Appendix G).

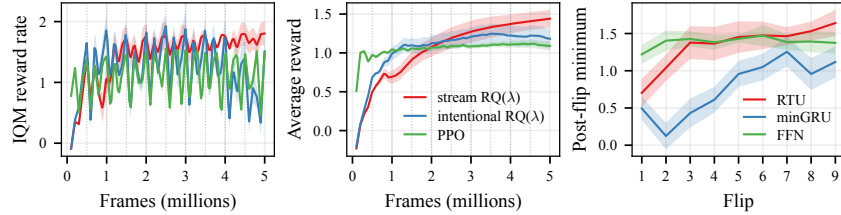


Figure 5: Continual foraging, 10 seeds. Left and centre compare the algorithms under RTU: the reward rate, an exponential moving average that tracks current performance, and the average reward over the agent’s lifetime, the statistic of Tang et al. (2026). Right compares the recurrences within stream RQ(λ): the minimum reward rate reached after each of the nine reward flips. Dotted lines mark the flips.

Memoryless control. The dashed curves are the same updates with the recurrence removed, at the same budget and seed count. Bootstrapping that difference at 5M, the recurrence is ahead on six of eight cells for stream RAC(λ), from +506 to +973, and on four of eight for intentional RAC(λ), from +314 to +774. Neither is behind on any cell, so the recurrence earns its place under both updates, and more reliably under the one whose step size transfers.

Result. Stream RAC(λ) and recurrent PPO are statistically indistinguishable on most of the benchmark (Figure 4). Bootstrapping the difference of means over the 10 seeds per cell separates only three of eight: PPO is ahead on Ant-V and Walker2d-P, stream RAC(λ) on Hopper-V, and the other five overlap. Appendix G lists the per-cell intervals. We therefore claim parity rather than an advantage.

Most of the difference from the setting these methods ship with is the trace decay: at $\lambda = 0.8$ stream RAC(λ) reaches 262 on Hopper-P against 1550 at 0.99, a factor of 5.9 (Appendix F). Walker2d resisted λ , a per-block step size, removing the entropy bonus and matching the network size, and responded only to the actor step size under the intentional update (Appendix G).

Intentional RAC(λ) does not complete the benchmark as published: on three cells its updates become non-finite partway through training. Appendix E attributes this to its preconditioner and shows that estimating it from the trace instead removes the divergence, after which all eight cells complete on every seed. Its actor step size does not transfer either. At $\eta_\pi = 0.01$ rather than the 0.05 of Sharifnassab et al. (2026) it improves on seven of eight cells and reaches parity with stream RAC(λ), ahead on one cell, behind on two and indistinguishable on five, and with recurrent PPO, ahead on two, behind on two and indistinguishable on four.

4.5 CONTINUAL FORAGING

A memoryless agent relearns after every flip, and the recurrent state takes the flips over.

Task and design. Foragax NeverEndingRelearning (Tang et al., 2026) is a grid world whose two biomes swap their mushroom rewards every 500k frames, so the agent must repeatedly abandon a learned foraging pattern and rediscover the other biome. Following Tang et al. (2026), the agent sees a 9×9 one-hot aperture, is never reset, and explores ϵ -greedily with a floor of 0.1. We run stream RQ(λ) and intentional RQ(λ) with RTU, minGRU and a memoryless network (FFN) under RTRL, and recurrent PPO with RTU at its tuned entropy coefficient, for 5M frames and 10 seeds (Figure 10). Tang et al. (2026) evaluate by average reward over the agent’s lifetime. We report both that and the exponential moving average of reward, which they also use, because the post-flip recoveries are what this section is about and a lifetime average integrates them away.

Result. The flips can be absorbed by relearning the weights or by carrying the phase in the state, and the post-flip minima separate the two. Stream RQ(λ) ends at a reward rate of 1.73 with RTU, 1.63 with the FFN and 1.47 with minGRU, against PPO at 1.16, a gap of 0.56 [0.08, 0.95]. Under the lifetime average of Tang et al. (2026) the ordering holds but the margin is unresolved, 1.42 against 1.10 [−0.05, 0.45]. The memoryless agent dips to between 1.2 and 1.4 at every flip and PPO to between 0.5 and 0.8, whereas the RTU dip shrinks from 0.70 at the first flip to 1.64 at the ninth (Figure 5), so the recurrent state progressively takes over the phase the FFN must relearn. The recurrence buys a shallower recovery rather than a higher ceiling, RTU and the FFN finishing at 1.73 and 1.63. Which update it is paired with matters more than the recurrence itself: RTU costs stream RQ(λ) nothing, +0.05 [−0.42, 0.40], and intentional RQ(λ) −0.81 [−1.22, −0.40], an interaction of 0.85 [0.25, 1.41]. Under intentional RQ(λ) the memoryless agent is the best agent here (Appendix E).

5 RELATED WORK

Streaming deep RL was made stable with feedforward networks by step-size bounds that cap the change in the prediction (Elsayed et al., 2024), pathwise actor updates that avoid a growing score function (Vasan et al., 2024), and intentional updates that solve for the step directly (Sharifnassab et al., 2026). We keep that machinery unchanged and add the recurrent channel. Appendix E shows all three of those step-size rules rest on an assumption the sensitivity trace violates. Exact RTRL (Williams & Zipser, 1989) is quartic in the hidden size, and has been made affordable either by approximating the influence matrix (Tallec & Ollivier, 2017; Menick et al., 2021; Bellec et al., 2020) or by restricting the recurrence so the exact influence is cheap, through diagonal complex cells (Orvieto et al., 2023; Zucchet et al., 2023; Elelimy et al., 2024) or element-wise gates (Irie et al., 2024). We take the second route and analyse what the resulting trace does inside a TD(λ) update, which Lemmel & Grosu (2025) do not. Recurrence is the standard remedy for partial observability, and Ni et al. (2022) give the baseline comparison our masked-MuJoCo protocol follows. Our continual-foraging evaluation sits in the tradition of tracking rather than converging (Sutton et al., 2007), where the failure mode is loss of plasticity (Dohare et al., 2024), which Foragax (Tang et al., 2026) induces with a periodically flipping reward.

6 DISCUSSION AND CONCLUSION

Exact RTRL over a diagonal recurrence substitutes for the truncated gradient in any streaming algorithm built on a per-step gradient and an eligibility trace, and the two traces compose into a credit horizon we predict, measure and follow from prediction through control. Acting on the part of that horizon the composition leaves short moves masked continuous control from a clear loss to parity.

The streaming agents trail recurrent PPO on eight of eleven POPGym tasks, three of which no method beats its memoryless counterpart. Both intentional variants diverge until their preconditioner is corrected, and on Foragax the best agent under that update is memoryless. Retention is necessary but not sufficient: on MemoryChain the credit horizon exceeds the solvable chain everywhere. Two limitations are structural, a single recurrent layer and ten seeds per cell.

REPRODUCIBILITY STATEMENT

Proposition 1 is stated in Section 3 and proved in Appendix L, with the staleness bound it relies on derived in Appendix N. Appendix K gives pseudocode for the recurrent streaming update and the architecture, and Appendix J lists every hyperparameter, network size and per-task setting, including the values taken unchanged from the works each algorithm comes from and the two we retuned. Every benchmark is public and unmodified apart from the observation masking described in Section 4.4. All results report seed counts in their captions, and every interval quoted in the text is a 95% bootstrap percentile interval over seeds with 20,000 resamples. The implementation is an open-source framework for memory-augmented reinforcement learning which we will release on acceptance, withheld here to preserve anonymity.

ETHICS STATEMENT

This work studies credit assignment in reinforcement learning agents on simulated control and memory benchmarks. It involves no human subjects, no personal or sensitive data, and no deployment on physical systems. We see no direct pathway to harm beyond the general dual-use considerations that apply to reinforcement learning methods, and we comply with the ICLR Code of Ethics.

USE OF LARGE LANGUAGE MODELS

Large language models were used as a general-purpose assistive tool, for code and experiment scaffolding, for aggregating and plotting results, and for editing prose. They were not used to generate research ideas, to derive or check the proofs, or to produce any experimental result. All claims, derivations and numbers in this paper were verified by the authors, who take full responsibility for the content.

REFERENCES

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11:3625, 2020.
- Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches, 2014. URL <https://arxiv.org/abs/1409.1259>.
- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmoood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.
- Esraa Elelimy, Adam White, Michael Bowling, and Martha White. Real-time recurrent learning using trace units in reinforcement learning. In *Advances in Neural Information Processing Systems*, 2024.
- Esraa Elelimy, Brett Daley, Andrew Patterson, Marlos C. Machado, Adam White, and Martha White. Deep reinforcement learning with gradient eligibility traces, 2025. URL <https://arxiv.org/abs/2507.09087>.
- Mohamed Elsayed, Gautham Vasan, and A. Rupam Mahmoood. Streaming deep reinforcement learning finally works. *arXiv preprint arXiv:2410.14606*, 2024.
- Leo Feng, Frederick Tung, Mohamed Osama Ahmed, Yoshua Bengio, and Hossein Hajimirsadeghi. Were rnns all we needed?, 2024.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

- Kazuki Irie, Anand Gopalakrishnan, and Jürgen Schmidhuber. Exploring the promise and limits of real-time recurrent learning. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=V2cBKtdC3a>.
- Julian Lemmel and Radu Grosu. Real-time recurrent reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.
- Jacob Menick, Erich Elsen, Utku Evci, Simon Osindero, Karen Simonyan, and Alex Graves. Practical real time recurrent learning with a sparse approximation to the Jacobian. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=q3KSThy2GwB>.
- Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. POPGym: Benchmarking partially observable reinforcement learning. In *The Eleventh International Conference on Learning Representations*, 2023.
- Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free RL can be a strong baseline for many POMDPs. In *International Conference on Machine Learning*, pp. 16691–16723. PMLR, 2022.
- Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences, 2023. URL <https://arxiv.org/abs/2303.06349>.
- Ian Osband, Yotam Doron, Matteo Hessel, John Aslanides, Eren Sezener, Andre Saraiva, Katrina McKinney, Tor Lattimore, Csaba Szepesvári, Satinder Singh, Benjamin Van Roy, Richard Sutton, David Silver, and Hado van Hasselt. Behaviour suite for reinforcement learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rygf-kSYwH>.
- Banafsheh Rafiee, Zaheer Abbas, Sina Ghiassian, Raksha Kumaraswamy, Richard S. Sutton, Elliot A. Ludvig, and Adam White. From eye-blinks to state construction: Diagnostic benchmarks for online representation learning. *Adaptive Behavior*, 31(1):3–19, 2023.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Arsalan Sharifnassab, Mohamed Elsayed, Kris De Asis, A. Rupam Mahmood, and Richard S. Sutton. Intentional updates for streaming reinforcement learning. *arXiv preprint arXiv:2604.19033*, 2026. URL <https://arxiv.org/abs/2604.19033>.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 1998.
- Richard S. Sutton, Anna Koop, and David Silver. On the role of tracking in stationary environments. In *Proceedings of the 24th International Conference on Machine Learning*, pp. 871–878, 2007.
- Corentin Tallec and Yann Ollivier. Unbiased online recurrent optimization, 2017.
- Steven Tang, Xinze Xiong, Anna Hakhverdyan, Andrew Patterson, Jacob Adkins, Jiamin He, Esraa Elelimy, Parham Mohammad Panahi, Martha White, and Adam White. Forager: a lightweight testbed for continual learning with partial observability in rl, 2026.
- Gautham Vasan, Mohamed Elsayed, Alireza Azimi, Jiamin He, Fahim Shahriar, Colin Bellinger, Martha White, and A. Rupam Mahmood. Deep policy gradient methods without batch updates, target networks, or replay buffers. In *Advances in Neural Information Processing Systems*, 2024.
- Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.
- Nicolas Zucchet, Robert Meier, Simon Schug, Asier Mujika, and João Sacramento. Online learning of long-range dependencies. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

A MEMORYCHAIN SOLVE RATES

Figure 1 reports the difference in solve rate between the two gradient computations. Figures 6, 7 and 8 give the underlying per-method rates for each of the three streaming algorithms, together with the difference. A run counts as successful when its final return exceeds 0.5; outlined cells mark, in each column, the longest chain at which the method still succeeds on a majority of seeds.

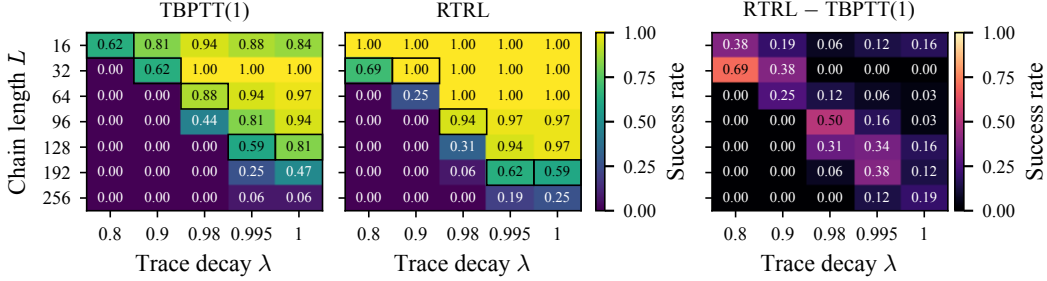


Figure 6: Solve rates for RQRC(λ) with a minGRU recurrence ($\gamma=0.999$, 16–32 seeds per cell). The solvable region under TBPTT(1) grows with the trace horizon, and exact RTRL pushes that boundary one grid step deeper at every λ but 0.9.

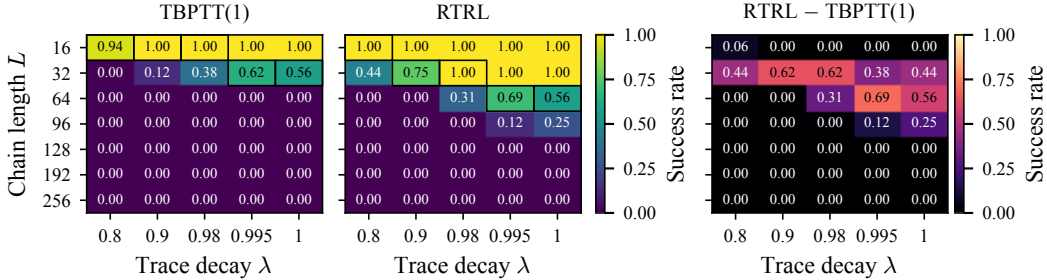


Figure 7: Solve rates for stream RQ(λ), which replaces RQRC(λ)'s corrected update with Overshooting-bounded Gradient Descent (ObGD) on a single Q -network (16 seeds per cell). The qualitative structure is unchanged but the reachable horizon is roughly half as long, and past $L=32$ TBPTT(1) fails at every λ while RTRL still solves a substantial fraction of seeds.

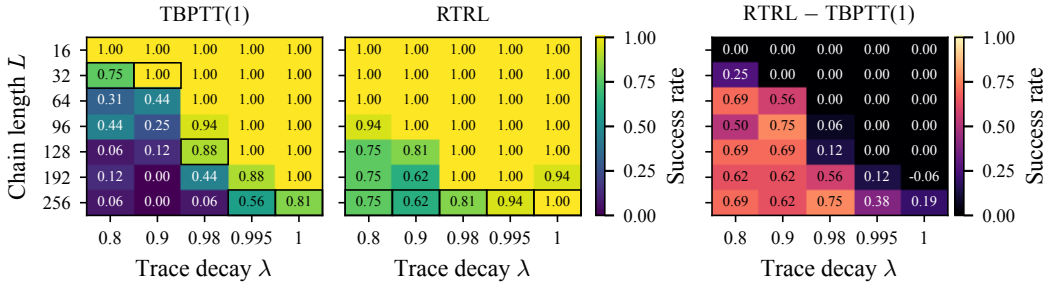


Figure 8: Solve rates for intentional RQ(λ) (16 seeds per cell). Exact RTRL solves a majority of seeds at every cell of the sweep, including $L=256$ at $\lambda=0.8$, so its reachable horizon is independent of λ over this range, while TBPTT(1) still tracks the trace horizon.

B CREDIT WEIGHT MEASUREMENTS IN DETAIL

B.1 PROTOCOL AND FULL MEASUREMENTS

Task and design. Proposition 1 predicts the weight each update places on the immediate Jacobian I_k of a step $n = T - k$ in the past, and (6) predicts how the two weights compare. Both are measurable. We train an agent with exact RTRL on MemoryChain with $L=64$ for 2×10^5 steps, freeze the parameters and roll out episodes, for 10 seeds per trace decay, cell and algorithm with 10 rollouts per seed. Because the recurrence is elementwise, both weights of Proposition 1 assemble exactly from the recorded head gradients c_t and diagonal state Jacobians J_t of a single rollout, and we fit the decay of $\|\omega_k\|$ against n and compare it with the predicted rates at $\lambda=1$ and $\lambda=0.98$, the two regimes of (6).

Result. Table 1 reports the measured rates. The TBPTT(1) weight decays at $\gamma\lambda$ to within 0.2% in every row, and the RTRL weight at $\max(\gamma\lambda, \rho)$ to within 2%, for both cells and both trace decays, confirming Proposition 1 directly. Repeating the measurement under stream $Q(\lambda)$ and intentional RQ(λ) gives the same agreement, so the decomposition holds across all three algorithms of Section 4.2.

Three further facts come out of the same measurements, detailed in Appendix B.1. The amplification is cell-dependent, 20 to 33 at $n=32$ for minGRU against 6 to 9 for RTU, and the cause is phase cancellation: minGRU’s coherence $\|\sum_i t_i\|/\sum_i \|t_i\|$ is exactly 1 at every lag because its retentions are real and positive, while RTU’s complex rotation drives it to 0.23–0.46 by $n=32$. Retention is learned for minGRU, which begins at $\rho=0.5$ and reaches 0.981 to 0.99995 depending on the algorithm, and initialised for RTU, which begins at 0.997. And exploration cuts favour RTRL: both implementations zero the eligibility trace on a non-greedy action but leave S_t intact, so a cut deletes every TBPTT(1) weight before it while the RTRL weight survives through the influence matrix, which on trained minGRU agents leaves TBPTT(1) with no credit at $n=32$ in 0.67 to 0.77 of rollouts at $\epsilon=0.1$. What the measurements do not establish is sufficiency: the credit horizon exceeds the observed solvable chain in every cell of every grid.

Table 1: Decay of the credit weight $\|\omega_k\|$ with lag on trained MemoryChain agents ($L=64$, medians over 10 seeds) against the rates of Proposition 1: $\gamma\lambda$ for TBPTT(1) and $\max(\gamma\lambda, \rho)$ for RTRL, with ρ the measured retention. The TBPTT(1) rate follows $\gamma\lambda$ across both trace decays while the RTRL rate does not; the last column is the amplification of exact RTRL over TBPTT(1) at lag 32.

Cell	λ	$\gamma\lambda$	ρ	$\ \omega^{\text{TBPTT}(1)}\ ^{1/n}$	$\ \omega^{\text{RTRL}}\ ^{1/n}$	Ratio at $n=32$
minGRU	1	0.9990	0.9993	0.9995	1.0140	20.1
minGRU	0.98	0.9790	0.9995	0.9791	0.9942	33.2
RTU	1	0.9990	0.9989	0.9972	1.0193	6.4
RTU	0.98	0.9790	0.9989	0.9786	1.0007	9.2

This appendix records the full per-lag measurements behind Table 1 and the discussion of Section 3.

Ratio growth and regimes. Table 1 reports the measured rates. The decay of $\omega_k^{\text{TBPTT}(1)}$ matches $\gamma\lambda$ at both trace decays, with the predicted value falling inside the interquartile range in each case, consistent with the first weight of Proposition 1. The measured ρ is 0.999 at both settings and is left unchanged by λ , which is what the second weight requires. The gated units that carry the cue saturate towards perfect retention, so lowering λ moves $\gamma\lambda$ away from one while leaving ρ where it is, and only the TBPTT(1) channel degrades. The weight under exact RTRL correspondingly decays at a rate close to one in both cases, within 1.5% of $\max(\gamma\lambda, \rho)$ at $\lambda=1$ and within 0.6% at $\lambda=0.98$.

The ratio of the two weights grows with the lag, reaching a median of 20 at $n=32$ for $\lambda=1$ and 33 for $\lambda=0.98$. This growth is roughly linear in n at both settings rather than separating into the two regimes of (6): at $\lambda=0.98$ the exponential form $(\rho/\gamma\lambda)^n$ evaluates to under 2 at $n=32$, far below what we measure, because the sum defining ω_k^{RTRL} accumulates on the order of n terms in either regime. The ratio is also the noisiest quantity we measure, spanning an order of magnitude across

seeds, so we draw no regime distinction from it. What is robust is its size: exact RTRL assigns between one and one and a half orders of magnitude more credit to parameter uses several tens of steps in the past, which is the quantity that the solve rates in Section 4.2 reflect.

Exploration cuts. The eligibility trace is zeroed on every non-greedy action and at episode boundaries, while the RTRL influence matrix is carried through unchanged. Writing m for the last cut at or before T , TBPTT(1) has $\omega_k = 0$ for every $k \leq m$, whereas $\omega_j^{\text{RTRL}} = \sum_{i=\max(j, m+1)}^T (\gamma\lambda)^{T-i} \mathbf{c}_i^\top \Phi_{i,j}$ remains non-zero for $j \leq m$. We measure the fraction of rollouts in which the TBPTT(1) weight at lag n is exactly zero while the RTRL weight is not, on trained minGRU agents at $L=64$, ten seeds and three rollouts per seed per ϵ .

Algorithm	λ	ϵ	$n=4$	$n=8$	$n=16$	$n=32$
stream RQ(λ)	1	0.01	0.00	0.00	0.07	0.13
stream RQ(λ)	1	0.1	0.23	0.27	0.50	0.67
stream RQ(λ)	0.98	0.01	0.03	0.03	0.03	0.07
stream RQ(λ)	0.98	0.1	0.17	0.23	0.57	0.77
RQRC(λ)	1	0.01	0.03	0.03	0.10	0.20
RQRC(λ)	1	0.1	0.23	0.27	0.53	0.73

Table 2: Fraction of rollouts in which a Watkins cut leaves TBPTT(1) with no credit at lag n while exact RTRL retains some, on minGRU at $L=64$.

C ADDITIONAL MEMORYCHAIN RESULTS

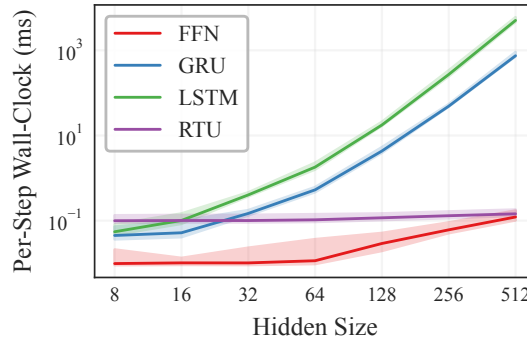


Figure 9: Per-step wall-clock time (ms) of a single gradient update as a function of hidden width, comparing feedforward backpropagation (FFN) against exact RTRL for GRU, LSTM, and RTU recurrent layers.

D ADDITIONAL FORAGAX RESULTS

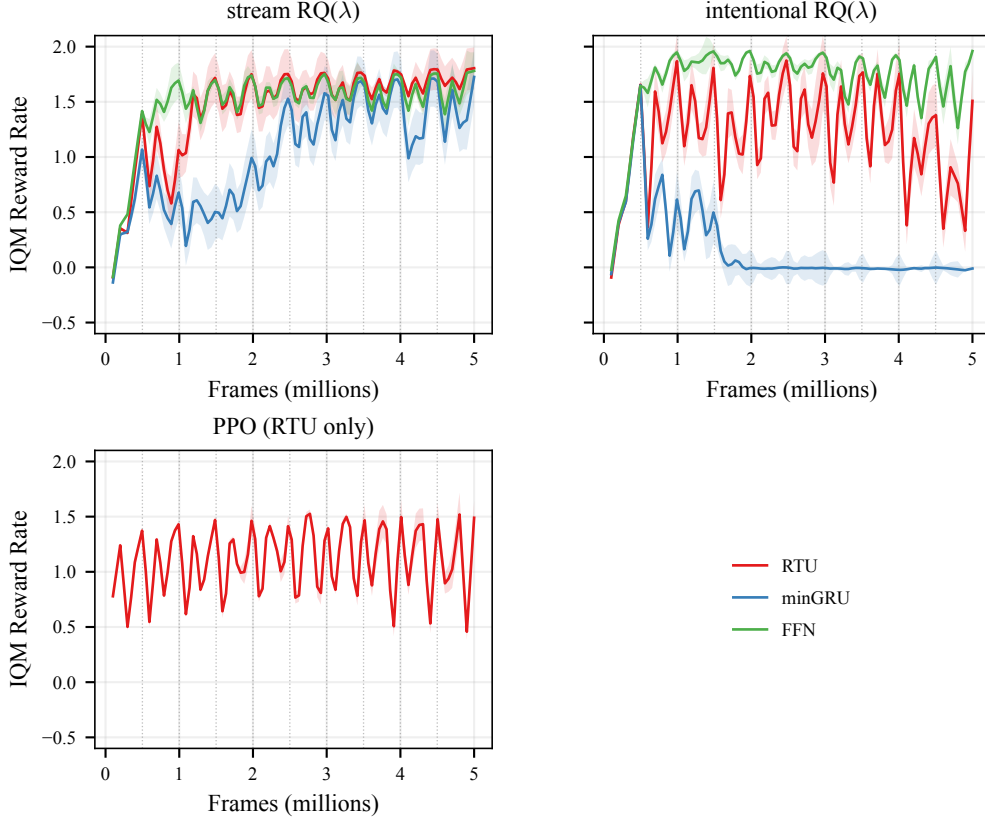


Figure 10: IQM reward rate, 10 seeds, on Foragax NeverEndingRelearning for stream $RQ(\lambda)$, intentional $RQ(\lambda)$ and batched PPO (entropy coefficient 0.1), with the minimum after each flip in the last panel (solid stream $RQ(\lambda)$, dashed PPO). Dotted lines mark the reward flips every 500k frames.

E STEP-SIZE RULES UNDER A SENSITIVITY TRACE

Every streaming method here sets its step size from a norm of the eligibility trace. That is sound when the trace is a decayed sum of recent gradients, because any coordinate carrying trace mass has recently carried gradient mass. Exact RTRL breaks the assumption: the sensitivity gives a coordinate trace mass on steps where its instantaneous gradient is negligible. We find the same assumption failing in three different rules.

ObGD bounds overshoot with an ℓ_1 norm. Its step is $\alpha / \max(1, \bar{\delta} \|z\|_1 \alpha \kappa)$ with $\bar{\delta} = \max(|\delta|, 1)$, which normalised rewards pin at exactly 1, so the step is set purely by $\|z\|_1$. Under RTRL the trace is dense and that norm grows through training: on masked Hopper the actor’s $\|z\|_1$ rises from 4.5×10^3 to 1.35×10^4 and its step falls from 8.9×10^{-5} to 3.2×10^{-5} , while the critic, whose trace stays flat, is unaffected. The mass is also badly distributed: measured on the masked-Hopper actor, the RTU block holds 91% of the parameters and 19% of the ℓ_1 gradient mass, while the policy head holds 2% of the parameters and 74% of the mass, so one small block sets the step for the whole network. Sweeping κ and η does not repair this (Appendix G), and neither does replacing the ℓ_1 norm with a per-block bound, which is worse on three of four cells at 5M.

Scale-free rules walk the retention. Adam takes a step of order its learning rate per parameter regardless of gradient magnitude, so it moves the recurrence’s retention parameter whether or not that gradient is informative. Training a pathwise actor-critic this way on masked HalfCheetah drives RTU’s ρ from 0.99949 to 0.99953; the steady-state gain $1/(1 - \rho)$ passes 2000, the cell integrates

a drifting observation for a thousand uninterrupted steps, and the carry overflows. Gradient clipping does not help, since Adam renormalises immediately after the clip.

Preconditioners estimated on the wrong quantity. The intentional update forms a per-parameter second moment from the gradient, $m = \sqrt{\hat{v}} + \epsilon$ with $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$, and then divides the trace by it, both in its denominator $\sqrt{\hat{\sigma} \sum z^2 / m}$ and in the update direction z / m . When the sensitivity puts mass into coordinates the gradient rarely visits, v stays near zero for those coordinates while z does not, and z^2 / m is inflated by a preconditioner that never saw them. The symptoms follow: on masked Hopper the actor step is sixteen times smaller than stream $\text{RAC}(\lambda)$'s at the same λ , and on three MuJoCo cells and on Foragax the updates become non-finite outright.

Trace preconditioning. The fix is to estimate the second moment from the trace, $v \leftarrow \beta_2 v + (1 - \beta_2) z^2$, so the preconditioner covers what it divides. Figure 11 shows the effect on Foragax, where intentional $\text{RQ}(\lambda)$ with RTU is otherwise dead from 700k frames onward: it recovers to a reward rate of about 1.7. The change also helps the memoryless agent, from 1.6 to 1.9, which says the mismatch is not purely an RTRL effect. An eligibility trace at $\lambda = 0.8$ already accumulates mass in rarely visited coordinates, and the sensitivity makes an existing problem acute. On masked MuJoCo the same change removes the divergence outright: intentional $\text{RAC}(\lambda)$ completes all eight cells on all ten seeds where three previously failed, and improves on seven of them.

It is not, however, a better preconditioner in general, and we would discourage adopting it as one. Where the original is stable it costs performance: on six POPGym tasks it is worse on every one, RepeatPrevious falling from 0.99 to 0.68, CountRecall from -0.53 to -0.96 and Battleship from -0.27 to -1.21 . On Foragax it is also a mitigation rather than a cure, since the RTU run still becomes non-finite late in training and minGRU recovers only to about 0.3. The pattern across the three benchmarks is that estimating the preconditioner from the trace helps exactly where the gradient-estimated one diverges and hurts where it does not.

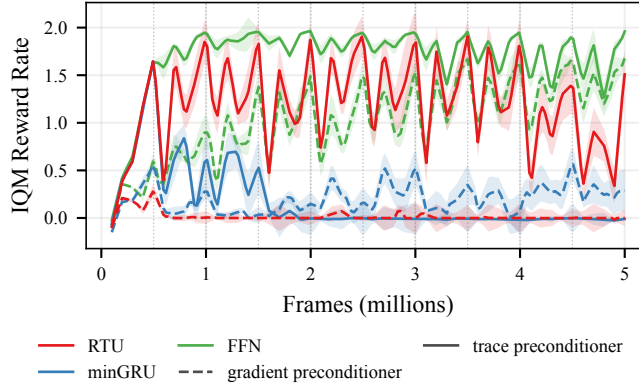


Figure 11: Intentional $\text{RQ}(\lambda)$ on Foragax with the preconditioner estimated from the trace (solid) and from the gradient (dashed), IQM over 10 seeds with standard error.

F TRACE DECAY ON MASKED MUJoCo

Proposition 1 gives the credit on a step k in the past a weight that decays at $\max(\gamma\lambda, \rho)$, and with RTU's initialisation $\rho \approx 0.997$, which invites reading λ as irrelevant. That reading holds only for the recurrent parameters. Their gradient carries the RTRL sensitivity, so their credit is a convolution of $(\gamma\lambda)^k$ and ρ^k and the slower mode wins. The feature extractor and the head have no sensitivity path, so their credit decays at $(\gamma\lambda)^k$ alone: a horizon of 4.8 steps at $\lambda = 0.8$ against the recurrence's 333. The composition therefore buys a long horizon for the recurrence and leaves the rest of the network short, and on a masked task the rest of the network is what has to learn what the inferred state is worth.

Table 3 sweeps λ on masked Hopper at 2M frames with 5 seeds. That cell was chosen because it had the largest gap to recurrent PPO under the $\lambda = 0.8$ configuration. At $\lambda = 0.99$ it becomes a win, so the selection was made under the setting the sweep then changed. Performance rises by a factor of 5.6 between $\lambda = 0.8$ and $\lambda = 0.99$ and falls again at $\lambda = 1$. The corresponding factor in the main experiment, at 5M frames with 10 seeds, is 5.9.

The composition explains why λ matters here at all, and does not predict where the optimum falls. It says the parameters the sensitivity reaches inherit ρ while the rest of the network decays at $\gamma\lambda$, so on a task whose difficulty is state inference the non-recurrent parameters are the binding constraint and lengthening their horizon should help; it does. It does not say the horizon should be lengthened without limit, and the data show it should not: $\lambda = 1$ brings $\gamma\lambda$ closer to ρ than $\lambda = 0.99$ does and performs worse, and the optimum sits at a horizon of 50 against a retention horizon of roughly 333. That turn is the ordinary bias-variance trade-off of $\text{TD}(\lambda)$, with an undecayed trace paying in variance what it gains in bias, and we report it as such rather than as a prediction of Proposition 1.

We use $\lambda = 0.99$ for masked MuJoCo throughout. Both source papers use 0.8, which was selected on fully observed control where no part of the network has to integrate an observation stream. We did not sweep PPO’s GAE λ , which stays at the 0.95 of Elelimy et al. (2024); a reader should weigh the comparison accordingly.

The value does not transfer. Sweeping the same three settings elsewhere places the optimum in a different place on every benchmark. On trace conditioning with RTU, the return error at ISI 20 is 0.036 at $\lambda = 0.9$, 0.032 at 0.95 and 0.045 at 0.99, and at ISI 40 0.090, 0.059 and 0.064, so 0.95 is best and 0.99 is worse. On POPGym the direction is algorithm-dependent. Raising λ to 0.99 from the value each method reports takes stream $\text{RQ}(\lambda)$ on RepeatPrevious from 0.05 to 0.93 in raw return, while it takes intentional $\text{RQ}(\lambda)$ on the same task from 0.83 to -0.12 and leaves $\text{RQRC}(\lambda)$ near its 0.95 value. What generalises is that an interior optimum exists and that it sits far below the retention horizon. Where it sits does not, and we report λ as a benchmark-level choice rather than a quantity the composition determines.

Table 3: Episode return at 2M frames on masked Hopper against the trace decay, mean over 5 seeds with standard error, stream $\text{RAC}(\lambda)$ with RTU under exact RTRL.

λ	$\gamma\lambda$	horizon $1/(1 - \gamma\lambda)$	return
0.8	0.792	4.8	223 ± 7
0.95	0.940	16.8	1041 ± 71
0.99	0.980	50.3	1255 ± 51
1.0	0.990	100.0	1089 ± 119

Lengthening the trace also changes what every streaming step-size rule does, because each normalises by a norm of that trace. ObGD bounds overshoot by $\bar{\delta}\|z\|_1$, so a longer trace shrinks its step roughly in proportion and the product of step and trace stays stable. The intentional update divides by $\sqrt{\sigma\|z\|^2}$ where σ itself grows with the trace, so it pays for the extra length twice: on masked Hopper its actor step falls from 5.5×10^{-6} at $\lambda = 0.8$ to 8.4×10^{-7} at $\lambda = 0.99$, sixteen times smaller than stream $\text{RAC}(\lambda)$ on the same task, and it recovers far less of the gap. Raising λ without revisiting the step-size constant is therefore not a safe default.

G ACTOR STEP-SIZE SWEEP ON MASKED MUJoCo

Table 4: Masked MuJoCo at 5M frames, 10 seeds. Bootstrap 95% percentile interval on the difference of means, stream RAC(λ) minus recurrent PPO, from 20,000 resamples. An interval excluding zero separates the two methods.

Cell	stream RAC(λ)	PPO	Difference	95% interval
Ant-P	2944	2605	+339	$[-74, 747]$
Ant-V	1437	1936	-499	$[-926, -63]$
HalfCheetah-P	2928	2449	+479	$[-237, 1182]$
HalfCheetah-V	2146	2742	-596	$[-1180, 14]$
Hopper-P	1550	1312	+238	$[-34, 513]$
Hopper-V	1214	1082	+132	$[1, 287]$
Walker2d-P	1073	1386	-313	$[-557, -27]$
Walker2d-V	1207	1330	-123	$[-426, 190]$

Under ObGD the effective actor step is $\alpha / \max(1, \alpha \kappa_\pi \bar{\delta} \|z\|_1)$, and with a Gaussian policy the score function grows as the policy sharpens, so $\|z\|_1$ grows and the step falls over training. Table 5 sweeps the actor constant of both methods in both directions on the two cells where the gap to recurrent PPO is widest, at 2M frames with 10 seeds and $\lambda = 0.99$, holding each critic constant at its default. The two methods behave differently. Stream RAC(λ) peaks at the $\kappa_\pi = 3$ it ships with and falls away on both sides, so its constant transfers to this setting. Intentional RAC(λ) does not: it peaks at $\eta_\pi = 0.01$, five times smaller than the published 0.05, gaining +429 on Hopper-P and +450 on Walker2d-P, while every increase above the default is worse. Section 4.4 reports it at the corrected value. The same argument that sets λ for the recurrence rather than inheriting it applies to the step size, and only one of the two published constants survives it.

Table 5: Final episode return at 2M frames on masked MuJoCo under actor step-size sweeps at $\lambda = 0.99$, mean over 10 seeds with standard error. Each method is bracketed on both sides of the constant its source paper uses.

Method	Actor constant	Hopper-P	Walker2d-P
stream RAC(λ)	$\kappa_\pi = 0.3$	176 ± 70	170 ± 42
	$\kappa_\pi = 1$	921 ± 167	717 ± 77
	$\kappa_\pi = 3$ (published)	1259 ± 62	925 ± 45
	$\kappa_\pi = 6$	1005 ± 59	715 ± 76
	$\kappa_\pi = 10$	809 ± 68	642 ± 88
intentional RAC(λ)	$\eta_\pi = 0.005$	1003 ± 58	1001 ± 40
	$\eta_\pi = 0.01$	1296 ± 78	1011 ± 47
	$\eta_\pi = 0.05$ (published)	867 ± 50	561 ± 40
	$\eta_\pi = 0.2$	324 ± 63	263 ± 44
	$\eta_\pi = 1$	168 ± 34	183 ± 38
recurrent PPO	—	1121 ± 50	1164 ± 49

H POPGYM WITH OTHER RECURRENCES

The recurrence is not held constant across benchmarks: MemoryChain uses minGRU, POPGym and masked MuJoCo use RTU, and trace conditioning reports both. We chose each from the cell that the corresponding source paper used, not from performance on the benchmark, but the choice matters: Section 4.2 reports that at $\lambda=1$ and $L=128$ a minGRU solves 60% of seeds where an RTU solves none. Figures 12 and 13 therefore repeat the POPGym sweep with a minGRU recurrence and with a memoryless network, at the same budget and seed count as the RTU figure in Section 4.3. The ordering among algorithms is largely preserved, but the cell matters where the task rewards accumulation: on MultiArmedBandit a minGRU under stream RQ(λ) reaches 0.76 against 0.55 for RTU, a gap of

+0.21 [+0.20, +0.22], and beats its own memoryless control by +0.13 where RTU falls 0.08 below it. Estimating the value of an arm is a running average, which real positive retentions accumulate coherently and a complex rotation does not, the same phase cancellation Section 3 measures. The memoryless comparison is what the per-task ceilings in Section 4.3 are computed from. We did not run masked MuJoCo or Foragax with a second recurrence, so the cell and the benchmark remain confounded there.

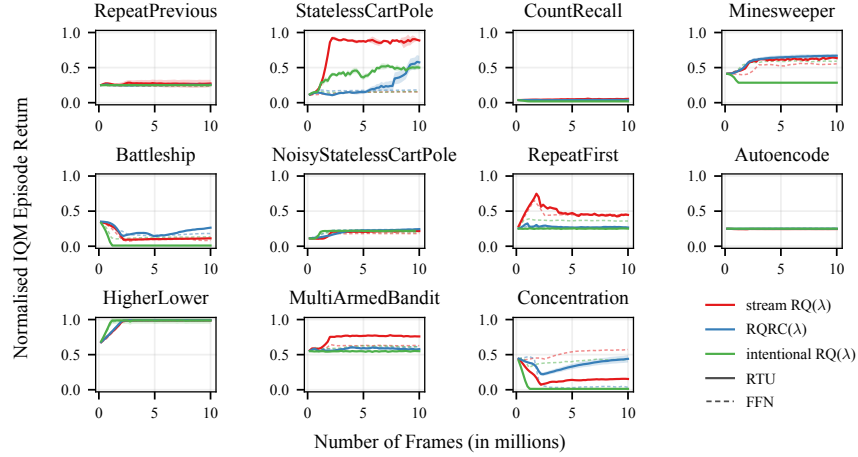


Figure 12: POPGym with a minGRU recurrence, otherwise identical to Figure 3. Normalised IQM episodic return over 10 seeds with standard error, with dashed curves giving the same algorithms with a feedforward network.

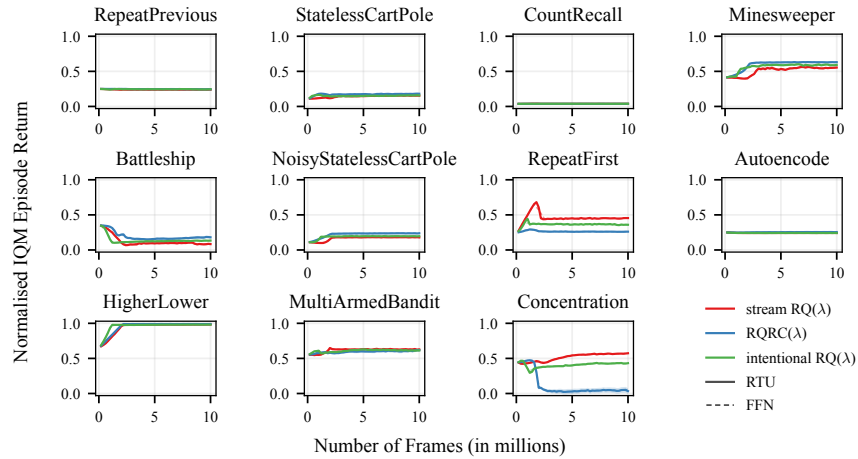


Figure 13: POPGym with a memoryless network, otherwise identical to Figure 3. These runs are what the per-task memoryless ceilings of Section 4.3 are estimated from on the two cartpole variants.

I FROZEN-RECURRENCE CONTROL

Table 6 reports the frozen-recurrence control referred to in Section 4.2: RQRC(λ) with TBPTT(1) on a cell whose recurrent parameters stay at initialisation. A random RTU with $\rho \approx 0.997$ and the eligibility trace solve $L=16$ in about half the seeds and nothing beyond $L=32$, and a frozen minGRU solves nothing, so the trained recurrence is required and a long-retention reservoir does not substitute for it.

Table 6: Mean MemoryChain score with the recurrent parameters frozen at initialisation, RQRC(λ) with TBPTT(1), 16 seeds.

Cell	λ	$L=16$	32	64	96	128	192	256
RTU	0.95	0.57	0.15	–	0.05	0.00	0.02	–0.02
RTU	1	0.55	0.27	0.01	0.02	0.01	–0.02	0.02
minGRU	0.95	0.08	0.02	0.01	–0.03	0.01	–0.06	–0.01
minGRU	1	–0.02	–0.01	–0.01	0.01	–	–0.03	0.04

J HYPERPARAMETERS

We used fully connected networks of width 64 with a single hidden block for MemoryChain and POPGym, and of width 128 with two hidden blocks for Masked MuJoCo, following Sharifnassab et al. (2026). All use LeakyReLU activations with LayerNorm (Ba et al., 2016) before each activation. On masked MuJoCo the PPO baseline is given the same body as the streaming agents, two width-128 blocks with LayerNorm, LeakyReLU and the same 90% sparse initialisation, and no projection after the recurrence, so that the comparison is at matched capacity: 139,405 parameters against 141,709, the difference being the streaming actor’s state-dependent standard deviation head. It keeps its own orthogonal head initialisations, which are a stability device rather than a capacity choice. On MemoryChain and POPGym it keeps its standard width-64 tanh network with orthogonal initialisation. A single RTU layer of hidden dimension 192 is inserted between the input and output blocks. We used separate networks for the policy and value functions. For stream RAC(λ) we used softmax policy parameterization in discrete control and a Gaussian with state-dependent standard deviation and actions clipped to $[-1, 1]$ in continuous control. For RQRC(λ) we used an ϵ -greedy policy. We used sparse initialization with a sparsity ratio of 90% (Elsayed et al., 2024) and normalized observations and rewards online with running statistics. stream RAC(λ) uses ObGD with step size $\alpha = 1$; the per-benchmark κ values for the policy and value networks are listed in the experiment-specific tables below. RQRC(λ) uses SGD with global-norm gradient clipping at 1.0 and learning rates 10^{-4} for the Q-network and 10^{-5} for the auxiliary h -network. For the PPO baseline we used the hyperparameters reported by Elelimy et al. (2024), including their real-time update: a rollout of 2048 steps, each stored with its own RTRL sensitivity, split into 32 minibatches of single steps.

Table 7: Hyperparameters for RQRC(λ) on the MemoryChain experiment (Section 4.2).

Name	Value
Chain length, L	{2, 4, 8, 16, 32, 48, 64, 128}
Frames per run	500,000
Number of seeds	16–32 per cell
Discount factor, γ	0.999
Trace decay, λ	0.95
Q -network step size, α_Q	10^{-4}
h -network step size, α_h	10^{-5}
Regularization coefficient, β	1.0
MLP width	64
RTU hidden dimension	192

Table 8: Hyperparameters for RQRC(λ) on POPGym (Section 4.3).

Name	Value
Frames per run	10,000,000
Number of seeds	10
Discount factor, γ	0.99
Trace decay, λ	0.95
Q -network step size, α_Q	10^{-4}
h -network step size, α_h	10^{-5}
Regularization coefficient, β	1.0
ε schedule	1.0 $\rightarrow$ 0.01 over 20% of training
MLP width	64
RTU hidden dimension	192

Table 9: Hyperparameters for stream AC(λ) on POPGym (Section 4.3).

Name	Value
Frames per run	10,000,000
Number of seeds	10
Discount factor, γ	0.99
Trace decay, λ	0.8
Entropy coefficient	0.095
Optimizer	ObGD
Actor step size, α_π	1.0
Critic step size, α_V	1.0
Actor κ_π	3.0
Critic κ_V	2.0
MLP width	64
RTU hidden dimension	192

Table 10: Hyperparameters for the Masked MuJoCo experiment (Section 4.4). Stream RAC(λ) and intentional RAC(λ) share the network, discount and trace decay and differ only in the optimizer.

Name	Value
Frames per run	5,000,000
Number of seeds	10
Observation regimes	masked velocities (P), masked positions (V)
Discount factor, γ	0.99
Trace decay, λ	0.99 (see Appendix F)
Entropy coefficient	0.01
MLP width (two blocks)	128
RTU hidden dimension	192
Stream RAC(λ) optimizer	ObGD
Actor step size, α_π	1.0
Critic step size, α_V	1.0
Actor κ_π	3.0
Critic κ_V	2.0
Intentional RAC(λ) optimizer	intentional update, trace-estimated preconditioner
Actor η_π	0.01 (see Appendix G)
Critic η_V	0.5
PPO network	matched to the streaming agents (see above)
PPO rollout length	2048
PPO single-step minibatches, epochs	32, 4
PPO GAE λ	0.95
PPO clip coefficient	0.2
PPO entropy coefficient	0.0
PPO step size (Adam), grad clip	10^{-4} , 0.5

Table 11: Hyperparameters for the trace-conditioning experiment (Section 4.1), following Rafiee et al. (2023).

Name	Value
Steps per run	2,000,000
Number of seeds	10
Nominal ISI and discount γ	10: 0.9, 20: 0.95, 40: 0.975, 80: 0.9875
ISI range	$[\text{ISI} - \lfloor \text{ISI}/3 \rfloor, \text{ISI} + \lfloor \text{ISI}/3 \rfloor]$
Inter-trial interval	$[80, 120]$ steps
Activation lengths (CS, US)	4, 2 steps
Trace decay, λ	$\{0.9, 1\}$
Return-error horizon	400 steps
Optimizer	ObGD, $\alpha = 1, \kappa = 2$
MLP width	64
Recurrent hidden dimension	192

Table 12: Hyperparameters for the Foragax NeverEndingRelearning experiment (Section 4.5). Stream RQ(λ) and intentional RQ(λ) share the network, discount, trace decay and exploration schedule and differ only in the optimizer.

Name	Value
Frames per run	5,000,000
Number of seeds	10
Reward-flip period	500,000 frames
Observation	9×9 aperture of one-hot object channels
Discount factor, γ	0.99
Trace decay, λ	0.8
Exploration, ε	$1.0 \rightarrow 0.1$ linearly over the first 10% of frames
Convolution	16 filters, 3×3 , valid padding
MLP width	128
Recurrent hidden dimension	192
Reward-rate smoothing	exponential average with $\beta = 10^{-4}$
Stream RQ(λ) optimizer	ObGD
Step size, α	1.0
κ	1.0
Intentional RQ(λ) optimizer	intentional update
η	0.1
PPO rollout length	2048
PPO single-step minibatches, epochs	32, 4
PPO GAE λ	0.95
PPO entropy coefficient	0.1, selected from $\{0.01, 0.1, 1\}$
PPO step size (Adam)	3×10^{-4} actor, 3×10^{-3} critic
PPO gradient clip	0.5

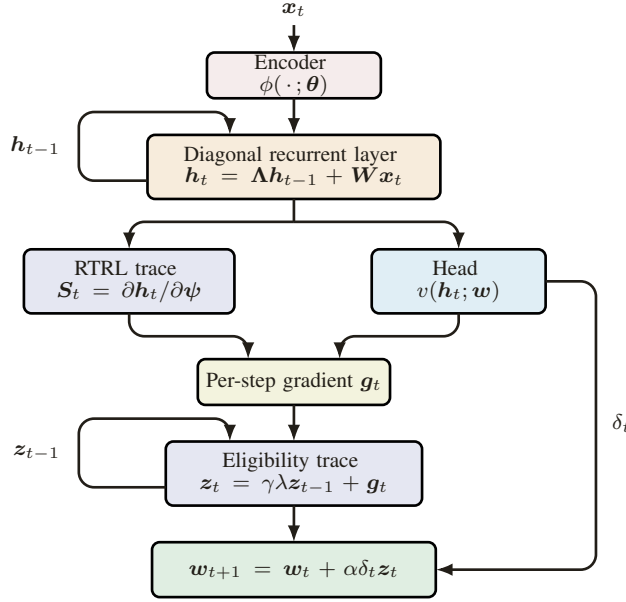
Table 13: Hyperparameters for stream RAC(λ) on the KMemoryChain experiment (Appendix M).

Name	Value
Memory length, K	$\{4, 8, 16\}$
Frames per run	300,000
Number of seeds	5
Discount factor, γ	0.99
Trace decay, λ	0.95
Actor step size, α_π	1.0
Critic step size, α_v	1.0
Actor ObGD κ_π	3
Critic ObGD κ_v	2.0
Entropy coefficient	0.01
RTU feature dimension	64
RTU hidden dimension	192

Table 14: Hyperparameters for RQRC(λ) on the KMemoryChain experiment (Appendix M).

Name	Value
Memory length, K	$\{4, 8, 16\}$
Frames per run	300,000
Number of seeds	5
Discount factor, γ	0.99
Trace decay, λ	0.95
Q -network step size, α_Q	10^{-4}
h -network step size, α_h	10^{-5}
Regularization coefficient, β	1.0
ϵ -greedy start	1.0
ϵ -greedy end	0.01
ϵ -greedy decay fraction	0.1
RTU feature dimension	64
RTU hidden dimension	192

K PSEUDOCODE

Figure 14: Data flow for one network with a value head. The recurrent layer’s RTRL trace and the head’s gradient form g_t , which the eligibility trace accumulates.

The contribution to either parent algorithm is the per-step gradient computation: each network maintains an RTU hidden state h and a forward-mode RTRL sensitivity matrix $S = \partial h / \partial \psi$, both advanced once per environment step, and gradients of any scalar depending on h are computed from S . Following the parent algorithms, θ , ψ and w denote each network’s full parameter set (encoder, RTU, head). Trace recursions and update rules are unchanged from the parents.

Algorithm 1 RQRC(λ) with RTU-RTRL

Input: a differentiable action-value function parametrization $\hat{q}_w$ with an RTU layer
 Input: a differentiable auxiliary function parametrization $\hat{h}_\theta$ with an RTU layer
 Algorithm parameters: learning rate α_q , h learning rate α_h , exploration parameter ϵ .
 Initialize $z_t^\theta \leftarrow \mathbf{0}$
 Initialize $z_t^w \leftarrow \mathbf{0}$
 Initialize $z_t^h \leftarrow \mathbf{0}$
 Initialize RTU carries $h_t^q, h_t^h \leftarrow \mathbf{0}$ and RTRL traces $S_t^q, S_t^h \leftarrow \mathbf{0}$
 Observe initial state S_0
for iteration $t = 1, 2, \dots$ **do**
 Advance (h_t^q, S_t^q) and (h_t^h, S_t^h) one RTU step on S_t .
 Sample an action $A_t \sim \pi$. ▷ We use an ϵ -greedy policy.
 Take action A_t , observe R_{t+1} and S_{t+1} .
 Compute δ_t and $\nabla_{w_t} \delta_t$ as in RQRC(λ) (Elelimy et al., 2025), with parameter gradients of $\hat{q}_w$ obtained from the RTRL trace S_t^q .
 Update the traces z_t^θ, z_t^w , and z_t^h as in RQRC(λ) (Elelimy et al., 2025), with parameter gradients of $\hat{h}_\theta$ obtained from the RTRL trace S_t^h .
 Compute Δw_t and $\Delta \theta_t$ as in RQRC(λ) (Elelimy et al., 2025).
 Update the parameters $w_{t+1} \leftarrow w_t + \alpha_q \Delta w_t$.
 Update the parameters $\theta_{t+1} \leftarrow \theta_t + \alpha_h \Delta \theta_t$.
 if episode terminated or A_t is non-greedy **then**
 Reset the traces z_t^θ, z_t^w , and z_t^h to zeros.
 end if
 if episode terminated **then**
 Reset the RTU carries h_t^q, h_t^h and RTRL traces S_t^q, S_t^h to zeros.
 end if
end for

Algorithm 2 stream AC(λ) with RTU-RTRL

Input: a differentiable policy parametrization $\pi(a \mid s; \theta)$ with an RTU layer
 Input: a differentiable state-value function parametrization $\hat{v}(s; w)$ with an RTU layer
 Algorithm parameters: discount γ , trace decay λ , policy step size α_π , value step size α_v , policy and value scaling factors κ_π, κ_v , entropy coefficient τ .
 Initialize $z_t^\theta \leftarrow \mathbf{0}$
 Initialize $z_t^w \leftarrow \mathbf{0}$
 Initialize RTU carries $h_t^\pi, h_t^v \leftarrow \mathbf{0}$ and RTRL traces $S_t^\pi, S_t^v \leftarrow \mathbf{0}$
 Initialize observation- and reward-normalization statistics as in stream AC(λ) (Elsayed et al., 2024)
 Observe initial state S_0
for iteration $t = 1, 2, \dots$ **do**
 Advance (h_t^π, S_t^π) and (h_t^v, S_t^v) one RTU step on S_t .
 Sample an action $A_t \sim \pi(\cdot \mid S_t, \theta_t)$.
 Take action A_t , observe R_{t+1} and S_{t+1} .
 Compute δ_t and the per-step gradients $\nabla_w \hat{v}(S_t, w_t)$ and $\nabla_\theta (\log \pi(A_t \mid S_t, \theta_t) + \tau \text{sign}(\delta_t) \mathcal{H}(\cdot \mid S_t, \theta_t))$ as in stream AC(λ) (Elsayed et al., 2024), with parameter gradients of $\hat{v}$ and π obtained from the RTRL traces S_t^v and S_t^π .
 Update the traces z_t^θ and z_t^w as in stream AC(λ) (Elsayed et al., 2024).
 Compute $\Delta \theta_t$ and Δw_t via ObGD as in stream AC(λ) (Elsayed et al., 2024).
 Update the parameters $\theta_{t+1} \leftarrow \theta_t + \Delta \theta_t$.
 Update the parameters $w_{t+1} \leftarrow w_t + \Delta w_t$.
 if episode terminated **then**
 Reset the traces z_t^θ and z_t^w to zeros.
 Reset the RTU carries h_t^π, h_t^v and RTRL traces S_t^π, S_t^v to zeros.
 end if
end for

COMPOSITION OF THE RTRL TRACE AND THE ELIGIBILITY TRACE

We prove Proposition 1. Throughout, $\mathbf{J}_t$ and $\mathbf{I}_t$ denote the state-to-state and immediate Jacobians of the recurrence as in Appendix N, and $\mathbf{c}_t^\top = \partial v(\mathbf{h}_t; \mathbf{w}_v) / \partial \mathbf{h}_t$.

THE TWO TRACES FORM ONE TRIANGULAR SYSTEM

Substituting the RTRL recursion $\mathbf{S}_t = \mathbf{J}_t \mathbf{S}_{t-1} + \mathbf{I}_t$ into the eligibility trace recursion $\mathbf{z}_t = \gamma \lambda \mathbf{z}_{t-1} + \mathbf{c}_t^\top \mathbf{S}_t$ shows that the pair evolves jointly as a single linear system,

$$\begin{bmatrix} \mathbf{S}_t \\ \mathbf{z}_t \end{bmatrix} = \begin{bmatrix} \mathbf{J}_t & \mathbf{0} \\ \mathbf{c}_t^\top \mathbf{J}_t & \gamma \lambda \end{bmatrix} \begin{bmatrix} \mathbf{S}_{t-1} \\ \mathbf{z}_{t-1} \end{bmatrix} + \begin{bmatrix} \mathbf{I}_t \\ \mathbf{c}_t^\top \mathbf{I}_t \end{bmatrix}. \quad (7)$$

The transition operator is block triangular, so its modes are those of the recurrence together with the scalar mode $\gamma \lambda$, and the coupling introduces no others. In the time-invariant case $\mathbf{J}_t \equiv \mathbf{J}$ its spectrum is exactly $\text{spec}(\mathbf{J}) \cup \{\gamma \lambda\}$, and the asymptotic decay of $\mathbf{z}_t$ is governed by the dominant mode $\max(\gamma \lambda, \rho)$ with ρ the spectral radius of $\mathbf{J}$.

Equation (7) also gives a precise reading of the two gradient computations. Exact RTRL retains the off-diagonal block $\mathbf{c}_t^\top \mathbf{J}_t$, which couples the accumulated sensitivity into the trace. TBPTT(1) sets that block to zero, decoupling $\mathbf{z}_t$ from $\mathbf{S}_{t-1}$ entirely and leaving $\gamma \lambda$ as the only surviving mode.

EXPLICIT CREDIT WEIGHTS

Unrolling (7) makes the resulting weights explicit. The RTRL recursion with $\mathbf{S}_{-1} = \mathbf{0}$ gives, by induction,

$$\mathbf{S}_i = \sum_{k=0}^i \Phi_{i,k} \mathbf{I}_k, \quad \Phi_{i,k} \doteq \mathbf{J}_i \mathbf{J}_{i-1} \cdots \mathbf{J}_{k+1}, \quad (8)$$

where $\Phi_{k,k}$ is the identity matrix, and the trace with $\mathbf{z}_{-1} = \mathbf{0}$ gives $\mathbf{z}_T = \sum_{i=0}^T (\gamma \lambda)^{T-i} \mathbf{g}_i$. Substituting each choice of per-step gradient and exchanging the order of summation,

$$\mathbf{z}_T^{\text{TBPTT}(1)} = \sum_{k=0}^T (\gamma \lambda)^{T-k} \mathbf{c}_k^\top \mathbf{I}_k, \quad \mathbf{z}_T^{\text{RTRL}} = \sum_{k=0}^T \left[\sum_{i=k}^T (\gamma \lambda)^{T-i} \mathbf{c}_i^\top \Phi_{i,k} \right] \mathbf{I}_k, \quad (9)$$

which are the weights of Proposition 1. Since $\Phi_{k,k}$ is the identity, the $i = k$ term of ω_k^{RTRL} is exactly $\omega_k^{\text{TBPTT}(1)}$: exact RTRL adds the contributions of all *future* states whose value depends on $\mathbf{h}_k$, and never removes credit that TBPTT(1) already assigns.

DECAY RATE

The weights are a discrete convolution of two geometric kernels, one with rate $\gamma \lambda$ and one with rate Φ . Writing $\rho \doteq \max_t \|\mathbf{J}_t\|$ and $C \doteq \max_t \|\mathbf{c}_t\|$ in any submultiplicative norm, $\|\Phi_{i,k}\| \leq \rho^{i-k}$ and, with $m = i - k$ and $n = T - k$,

$$\|\omega_k^{\text{RTRL}}\| \leq C \sum_{m=0}^n (\gamma \lambda)^{n-m} \rho^m = C \frac{(\gamma \lambda)^{n+1} - \rho^{n+1}}{\gamma \lambda - \rho} = \mathcal{O}(\max(\gamma \lambda, \rho)^n) \quad (10)$$

for $\rho \neq \gamma \lambda$, which proves the proposition and recovers the dominant mode of (7). The convolution of two geometric sequences decays at the rate of the slower one, giving the two regimes: for $\rho < \gamma \lambda$ the bound is $\mathcal{O}((\gamma \lambda)^n)$, the same order as TBPTT(1), so maintaining $\mathbf{S}_t$ exactly changes the bound on the decay by at most a constant. For $\rho > \gamma \lambda$ it is $\mathcal{O}(\rho^n)$, larger than the TBPTT(1) weight by a factor $(\rho/\gamma \lambda)^n$. In the boundary case $\rho = \gamma \lambda$, encountered when a gated recurrence saturates towards perfect retention, the sum has $n + 1$ terms of equal size and the bound carries an extra factor of $n + 1$. These are upper bounds throughout: $\|\Phi_{i,k}\| \leq \rho^{i-k}$ gives no matching lower bound on a sum whose terms may cancel, and Section 3 measures exactly such cancellation for RTU, whose coherence falls to 0.23. The measured decay rates are therefore consistent with the bound rather than a confirmation of a predicted rate. Collecting the three cases gives (6).

L.4 WHAT THE DECAY RATE DOES AND DOES NOT SAY

Equation (6) compares the rate at which the two weights on $\mathbf{I}_k$ decay with lag. Three qualifications are needed before reading it as a statement about attainable memory.

First, the two weights multiply the same Jacobians but are assembled from different sensitivities. $\omega_k^{\text{TBPTT}(1)}$ contains only c_k , the sensitivity of the value *at step k* to the immediate parameter use at that step, whereas ω_k^{RTRL} additionally contains c_i for $i > k$ weighted by $\Phi_{i,k}$, the dependence of *future* values on $\mathbf{h}_k$. Only the latter is propagation of credit through the state. A large trace weight therefore does not by itself substitute for the recurrent gradient: it does so only when the intermediate values are already informative about the eventual dependency, so that shaping c_k is sufficient. Diagnostics in which the value at the cue step must itself predict the terminal reward have this property, whereas tasks in which intermediate values carry no such signal do not.

Second, the bound governs how credit is weighted, not whether there is anything to credit. If the recurrence does not retain state, $\Phi_{T,0}$ is negligible and $\mathbf{h}_T$ is independent of $\mathbf{h}_0$; no choice of λ recovers a dependence that the forward pass never formed. The trace credits a memory path that the architecture supplies, and ρ appears in (6) in both roles at once.

Third, λ is not free. The undecayed trace at $\lambda = 1$ is the Monte Carlo return, whose variance grows with the number of stochastic transitions it accumulates. On a deterministic diagnostic with a single terminal reward this cost is negligible, which is why $\lambda = 1$ is admissible there; in stochastic or continuing tasks it is not, and $\gamma\lambda$ is bounded well below one for reasons unrelated to credit assignment. The regime $\rho > \gamma\lambda$, in which exact RTRL extends the horizon, is therefore the typical rather than the exceptional case.

Remark. The bound is stated in terms of $\rho = \max_t \|\mathbf{J}_t\|$ and is therefore loose for recurrences whose state Jacobians are not simultaneously diagonalisable with non-negative spectrum. For a complex-diagonal recurrence such as an RTU, $\Phi_{i,k}$ is a product of scaled rotations, and the terms of ω_k^{RTRL} may interfere destructively in phase even when ρ is close to one. The attainable horizon is then governed by the phase-aligned component of the product rather than by its norm.

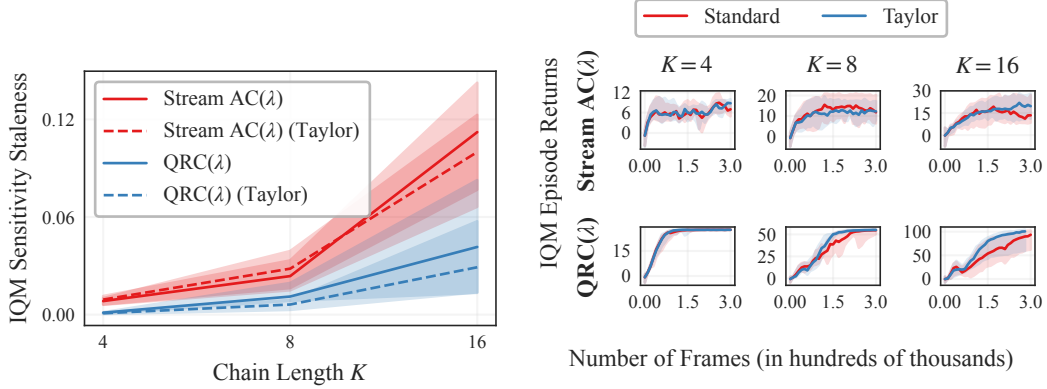
M RTRL STALENESS ON KMEMORYCHAIN

The RTRL sensitivity matrix $\mathbf{S}_t = \partial \mathbf{h}_t / \partial \psi$ is built up by accumulating immediate Jacobians $\mathbf{I}_k(\psi_k)$ injected at past steps and propagated forward by the state-to-state Jacobian $\mathbf{J}_t$. Under a streaming update the parameters ψ move every step, so each $\mathbf{I}_k(\psi_k)$ in this sum was evaluated at parameters the optimizer has since left behind. The maintained sensitivity is therefore a biased estimate of the ideal sensitivity $\mathbf{S}_t^*$, which re-evaluates every historical immediate Jacobian at the current parameters ψ_t .

RTRL staleness has been observed qualitatively in prior work (Williams & Zipser, 1989; Menick et al., 2021; Irie et al., 2024; Elelimy et al., 2024), with smaller learning rates (Menick et al., 2021) or periodic recomputation of the trace over the trajectory (Elelimy et al., 2024) suggested as mitigations, but it has not been quantitatively bounded.

Appendix N bounds this staleness in steady state and shows that a first-order Taylor correction in ψ reduces the error from $\mathcal{O}(\eta)$ to $\mathcal{O}(\eta^2)$. This section probes the bias empirically and tests whether the correction tracks the bound under sustained credit-assignment load.

Task and design. We use KMemoryChain, an every-step variant of MemoryChain designed to exercise the RTRL trace continuously. At each step the agent observes a fresh ± 1 bit together with a time-remaining feature, and is rewarded ± 1 for predicting the bit it saw K steps ago. Rewards activate after a K -step warmup. The classic MemoryChain rewards a single recall at episode end and leaves the immediate Jacobian inactive for most of the rollout, which is precisely the regime in which staleness has the least opportunity to accumulate. KMemoryChain instead injects a new bit and emits a reward-relevant target at every step, keeping the immediate Jacobian non-zero throughout the rollout and making K a direct dial on the credit-assignment horizon over which the sensitivity must remain accurate. We sweep $K \in \{4, 8, 16\}$ and run both streaming algorithms, RQRC(λ) and



(a) RTRL sensitivity staleness on KMemoryChain. Relative ℓ_2 distance between the online sensitivity matrix $\tilde{S}_t$ and its recomputed reference S_t^* , $\|\tilde{S}_t - S_t^*\|_2 / \|S_t^*\|_2$, over different chain lengths. (b) IQM episodic return over 5 seeds with shaded standard error on different chain lengths of KMemoryChain.

stream RAC(λ). Each algorithm is run with the standard RTRL sensitivity and with the first-order Taylor correction of Appendix N enabled, giving four configurations per K .

Staleness metric. At every training step we maintain a buffer of the observations from the current episode. To form a reference, we replay the buffer forward from the saved initial state using the *current* parameters ψ_t , recomputing what the sensitivity would be if every immediate Jacobian in the episode had been evaluated at ψ_t rather than at the parameters in force when it was originally injected. The staleness reported is the ℓ_2 distance between the live sensitivity carried by the algorithm and this recomputed reference.

Result. Figure 15a reports the IQM sensitivity staleness on KMemoryChain as a function of the memory credit-assignment horizon K . Staleness rises with K across all four configurations, remaining near zero at $K=4$ and growing through $K=8$ to its peak at $K=16$. This K -dependence is consistent with the $(1 - \rho)^{-2}$ scaling of the steady-state bound in Equation (16): longer credit-assignment horizons require the recurrence to retain memory over more steps, raising the effective contraction constant and amplifying the bound.

The two algorithms differ in absolute staleness level: stream RAC(λ) staleness is consistently higher than RQRC(λ) staleness and grows more steeply with K . We attribute this gap to the optimizer rather than the algorithm. Stream RAC(λ) uses ObGD (Elsayed et al., 2024) with base step size $\alpha = 1$, which is scaled down only when the effective-step-size threshold is violated and otherwise drives large per-step parameter updates. RQRC(λ), by contrast, uses plain SGD with fixed learning rates of 10^{-4} for the Q-network and 10^{-5} for the auxiliary h -network, two to three orders of magnitude smaller than the *nominal* ObGD step, though Appendix E measures ObGD’s *effective* step on masked Hopper at around 10^{-4} , the same order as RQRC(λ)’s; the mechanism we suggest here is therefore not established by the step sizes alone. Because the steady-state bound in Equation (16) scales linearly with the per-step parameter update magnitude ηG , the larger ObGD updates translate directly into proportionally larger steady-state staleness. The Taylor correction reduces staleness across both algorithms, with the largest absolute reduction at $K=16$ where the uncorrected staleness is largest and the leading-order term predicted by Equation (15) dominates. At $K=4$ the corrected and uncorrected variants are essentially indistinguishable, leaving little headroom for a first-order correction to act on.

Figure 15b reports the corresponding IQM episodic returns across training for each (algorithm, K) cell. The Taylor correction does not degrade learning at any K , and at $K=4$ and $K=8$ it tracks the standard sensitivity throughout training. At $K=16$, where the standard sensitivity incurs the largest staleness, the Taylor correction appears to learn faster and produce a more stable training curve, providing downstream evidence that the staleness reductions in Figure 15a translate into

improved credit assignment in the regime where the bound is tightest. Taken together, streaming RTRL staleness stays bounded under sustained credit-assignment load, scales with K in the direction the bound predicts, and tightens under the Taylor correction in the regime where the leading-order term dominates.

N STALENESS ERROR BOUND FOR THE RTRL SENSITIVITY MATRIX

In the nonlinear setting, the RTRL sensitivity matrix maintained by an RTU layer incurs a *staleness error*: immediate Jacobians accumulated at earlier steps were evaluated at parameters that the optimizer has since moved away from. We bound this error and give a second-order correction that reduces it from $\mathcal{O}(\eta)$ to $\mathcal{O}(\eta^2)$.

N.1 SETUP AND ASSUMPTIONS

For a recurrent network $\mathbf{h}_t = \mathbf{f}(\mathbf{h}_{t-1}, \mathbf{x}_t; \boldsymbol{\psi})$ with state-to-state Jacobian $\mathbf{J}_t = \partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$ and immediate Jacobian $\mathbf{I}_t(\boldsymbol{\psi}) = \partial \mathbf{h}_t / \partial \boldsymbol{\psi}$, RTRL maintains the sensitivity matrix

$$\mathbf{S}_t = \mathbf{J}_t \mathbf{S}_{t-1} + \mathbf{I}_t. \quad (11)$$

Let $\boldsymbol{\psi}_t \in \mathbb{R}^p$ denote the recurrent parameters in force at step t . The *stale sensitivity matrix* maintained by the online algorithm is

$$\tilde{\mathbf{S}}_t = \mathbf{J}_t \tilde{\mathbf{S}}_{t-1} + \mathbf{I}_t(\boldsymbol{\psi}_t) = \sum_{k=0}^t \left(\prod_{j=k+1}^t \mathbf{J}_j \right) \mathbf{I}_k(\boldsymbol{\psi}_k), \quad (12)$$

where the immediate Jacobian injected at step k was evaluated at the parameters $\boldsymbol{\psi}_k$ in force at that step. The *ideal sensitivity matrix*, which re-evaluates each historical immediate Jacobian at the current parameters, is

$$\mathbf{S}_t^* = \sum_{k=0}^t \left(\prod_{j=k+1}^t \mathbf{J}_j \right) \mathbf{I}_k(\boldsymbol{\psi}_t). \quad (13)$$

The staleness error is $E_t = \|\mathbf{S}_t^* - \tilde{\mathbf{S}}_t\|$.

Assumptions.

- (A1) **Non-expansion.** $\|\mathbf{J}_t\| \leq \rho \leq 1$ uniformly in t .¹
- (A2) **Bounded updates.** $\|\boldsymbol{\psi}_t - \boldsymbol{\psi}_{t-1}\| \leq \eta G$ for all t , where η is a step size and G bounds the per-step update magnitude.
- (A3) **Lipschitz immediate Jacobian.** The map $\boldsymbol{\psi} \mapsto \mathbf{I}_t(\boldsymbol{\psi})$ is Lipschitz with constant L_I , uniformly in t : $\|\mathbf{I}_t(\boldsymbol{\psi}) - \mathbf{I}_t(\boldsymbol{\psi}')\| \leq L_I \|\boldsymbol{\psi} - \boldsymbol{\psi}'\|$.
- (A4) **Bounded magnitude.** $\|\mathbf{I}_t(\boldsymbol{\psi})\| \leq M_I$ uniformly.

Bound on the sensitivity matrix. Taking norms in the stale recursion and applying the triangle inequality, $\|\tilde{\mathbf{S}}_t\| \leq \rho \|\tilde{\mathbf{S}}_{t-1}\| + M_I$, which unrolls to the geometric steady-state bound $\|\tilde{\mathbf{S}}_t\| \leq M_I / (1 - \rho)$.

Error recurrence. Expanding $\mathbf{S}_{t+1}^*$ and $\tilde{\mathbf{S}}_{t+1}$, the injection at step $t+1$ cancels because both contain $\mathbf{I}_{t+1}(\boldsymbol{\psi}_{t+1})$. Adding and subtracting $\mathbf{I}_k(\boldsymbol{\psi}_t)$ inside the bracket of the resulting difference and factoring out the leading Jacobian $\mathbf{J}_{t+1}$,

$$\mathbf{S}_{t+1}^* - \tilde{\mathbf{S}}_{t+1} = \mathbf{J}_{t+1} \sum_{k=0}^t \left(\prod_{j=k+1}^t \mathbf{J}_j \right) [\mathbf{I}_k(\boldsymbol{\psi}_{t+1}) - \mathbf{I}_k(\boldsymbol{\psi}_t)] + \mathbf{J}_{t+1} (\mathbf{S}_t^* - \tilde{\mathbf{S}}_t). \quad (14)$$

¹For an RTU layer with $\boldsymbol{\Lambda} = \text{diag}(r_k e^{i\theta_k})$ the constant is $\rho = \max_k r_k$, kept at most one by the standard RTU parameterization. A gated recurrence that learns to retain its state approaches the boundary $\rho = 1$, which the steady-state argument below excludes and the finite-horizon bound of Appendix N.2 admits.

Taking norms, applying (A1) to each product of Jacobians, applying (A2) and (A3) to the freshly injected term, and bounding the geometric sum by $1/(1 - \rho)$,

$$E_{t+1} \leq \rho \cdot \frac{L_I \eta G}{1 - \rho} + \rho E_t. \quad (15)$$

This is a first-order linear recurrence with contraction factor $\rho < 1$ and constant driving term. The transient $\rho^t E_0$ vanishes and the geometric series converges, yielding the steady-state bound

$$E_\infty \leq \frac{\rho L_I G \eta}{(1 - \rho)^2}. \quad (16)$$

Periodic updates. If ψ is held fixed for m steps between updates, the staleness injected at each update decays by ρ^m before the next, sharpening the bound to

$$E_\infty \leq \frac{\rho L_I G \eta}{(1 - \rho)(1 - \rho^m)}. \quad (17)$$

Since $1 - \rho^m \geq 1 - \rho$ for $m \geq 1$, periodic updates strictly reduce the steady-state staleness.

N.2 A FINITE-HORIZON BOUND WITHOUT CONTRACTION

The steady-state bound (16) requires $\rho < 1$ and diverges as $\rho \rightarrow 1$. This excludes the case of interest for a gated recurrence that learns to retain its state, where the retention factor of the units carrying the signal approaches one. The episodic setting, however, bounds the accumulation horizon by the episode length, so no contraction is needed if the error is bounded over a finite horizon rather than in steady state.

Fix an episode of length T and write $n = T - k$ for the lag of the contribution injected at step k . Telescoping (A2) gives $\|\psi_T - \psi_k\| \leq \eta G(T - k)$, so applying (A3) to each term of the difference between the ideal and stale matrices, and (A1) to each product of Jacobians,

$$E_T = \left\| \sum_{k=0}^T \left(\prod_{j=k+1}^T \mathbf{J}_j \right) [\mathbf{I}_k(\psi_T) - \mathbf{I}_k(\psi_k)] \right\| \leq L_I \eta G \sum_{n=0}^T n \rho^n, \quad (18)$$

which is finite for every $\rho \leq 1$ and every finite T . Evaluating the sum,

$$E_T \leq L_I \eta G \frac{\rho(1 - (T + 1)\rho^T + T\rho^{T+1})}{(1 - \rho)^2}, \quad \rho < 1, \quad (19)$$

and taking $T \rightarrow \infty$ recovers (16), so the finite-horizon bound strictly generalises the steady-state one. At the boundary $\rho = 1$ the sum evaluates to $T(T + 1)/2$ and

$$E_T \leq \frac{1}{2} L_I \eta G T(T + 1), \quad (20)$$

so a recurrence that retains its state perfectly still incurs staleness that is bounded on any finite episode, growing at most quadratically in its length rather than diverging. The two factors of T have distinct origins: one counts the contributions accumulated in the sensitivity matrix, and one measures how far the parameters have moved since the oldest of them was injected.

Remark. The bound (18) sums the magnitudes of the per-step errors and is therefore tight only when those errors align. When the increments $\mathbf{I}_k(\psi_T) - \mathbf{I}_k(\psi_k)$ are not systematically aligned across k , their sum grows more slowly than the sum of their norms, and the dependence on T is correspondingly weaker.

N.3 A FIRST-ORDER TAYLOR-TRACE CORRECTION

The leading-order term in the error recurrence (15) is the freshly injected staleness $\mathbf{I}_k(\psi_{t+1}) - \mathbf{I}_k(\psi_t)$. A first-order Taylor expansion in ψ gives

$$\mathbf{I}_k(\psi_t) \approx \mathbf{I}_k(\psi_{t-1}) + \left. \frac{\partial \mathbf{I}_k}{\partial \psi} \right|_{\psi_{t-1}} \Delta \psi_{t-1}, \quad (21)$$

where $\Delta\psi_{t-1} = \psi_t - \psi_{t-1}$. Substituting into S_t^* and grouping the second-order factors into an auxiliary accumulator

$$\Omega_t = J_t \Omega_{t-1} + \left. \frac{\partial \mathbf{I}_t}{\partial \psi} \right|_{\psi_t} \quad (22)$$

yields a corrected update

$$S_t = J_t (S_{t-1} + \Omega_{t-1} \Delta\psi_{t-1}) + \mathbf{I}_t(\psi_t). \quad (23)$$

The additive term $J_t \Omega_{t-1} \Delta\psi_{t-1}$ neutralizes the leading-order staleness injection, reducing the residual error from $\mathcal{O}(\|\Delta\psi\|)$ to $\mathcal{O}(\|\Delta\psi\|^2)$.

Storing Ω_t exactly is intractable: since $\mathbf{I}_t$ has shape $|h| \times p$, Ω_t is a rank-three tensor of shape $|h| \times p \times p$ requiring $\mathcal{O}(|h|p^2)$ memory. A parameter-wise diagonal approximation,

$$\omega_t = J_t \omega_{t-1} + \text{diag}_{\psi} \left(\frac{\partial \mathbf{I}_t}{\partial \psi} \right), \quad S_t = J_t (S_{t-1} + \omega_{t-1} \odot \Delta\psi_{t-1}) + \mathbf{I}_t(\psi_t), \quad (24)$$

recovers the original space complexity of the RTRL sensitivity, where $\odot$ denotes the elementwise product along the parameter axis.